VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Virtual Museum for the Development and Preservation of Cai Luong's Intangible Cultural and Artistic Heritage

By
Do Nguyen Chuong - ITITIU22021

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Computer Science

Ho Chi Minh City, Vietnam
2026

Virtual Museum for the Development and Preservation of Cai Luong's Intangible Cultural and Artistic Heritage

APPROVED BY:

Le Duy Tan, Ph.D., supervisor

THESIS COMMITTEE

Le Duy Tan, Ph.D

.....

Acknowledgments

I am profoundly grateful to my thesis supervisor, Dr. Le Duy Tan, whose expertise and mentorship have guided this research from inception to completion. His insightful guidance, invaluable support, and advice not only shaped the theoretical and practical dimensions of this study but also inspired me to approach complex challenges with confidence and clarity. Dr. Tan's encouragement and constructive feedback provided a robust foundation for refining the research methodology and academic rigor, which played an essential role in the successful completion of this thesis.

I would also like to express my appreciation to lecturers at the School of Computer Science and Engineering, whose expert guidance in technical fields and application development significantly contributed to the development of the VCaiLuong microservice system on various platforms.

I would also like to thank all the staff in the International University who have contributed to my educational journey. Their commitment to excellence in education and their dedication to fostering a supportive and intellectually stimulating environment have greatly enriched my learning experience, providing me with the skills and inspiration necessary to undertake this research.

Additionally, I am also grateful to my classmates and my colleagues for their support, collaboration, and helpful discussions during the research process. Their encouragement and belief in this project provided a strong foundation throughout the research process, fostering a collaborative environment that enriched the development of this thesis. Their contributions, whether through brainstorming sessions or moral support, were invaluable in sustaining my motivation and ensuring the success of this endeavor.

Lastly, I would like to thank my family for their unconditional love, patience, and moral support during my academic journey. Without their belief in me, this achievement would not have been possible.

This thesis would not have been complete without the guidance and support from all the individuals mentioned above. I am deeply grateful and wish to extend my heartfelt thanks to all who have made sacrifices and contributed to the realization of this thesis.

Table of Contents

List of Tables

List of Figures

List of Abbreviations

VICAST	Vietnam Institute of Culture, Arts and Sports
AI	Artificial Intelligence
VR	Virtual Reality
AR	Augmented Reality
DHC	Digital Heritage Collection
UI/UX	User Interface / User Experience
VNFAM	Vietnam National Fine Arts Museum
3D	Three-Dimensional
CDN	Content Delivery Network
PUB/SUB	Publish–Subscribe
DDD	Domain-Driven Design
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
SSO	Single Sign On
JWT	JSON Web Token
TTL	Time To Live
SAS	Shared Access Signature

Abstract

Flood information often appears first as a short message, a photo, or a location shared by someone already near the water. That first signal is useful, but it becomes difficult to verify and manage when many reports arrive through separate channels. This thesis presents VietFlood, a multi-platform prototype for collecting citizen flood reports and giving relief users and administrators a clearer place to review them.

VietFlood is built as four connected parts. The backend uses NestJS with an API gateway, an authentication service, a reports service, and a Socket.IO tracking gateway. RabbitMQ carries messages between services. PostgreSQL stores users, refresh tokens, and flood reports through TypeORM. Redis is used for cached report lookup. Cloudinary stores uploaded evidence files, while the shared `vietflood_common` package keeps logging, Redis access, and Cloudinary helpers in one place. The mobile application is developed with Expo React Native for citizens and relief workers. The web dashboard is developed with Next.js, React, TypeScript, and Tailwind CSS for administrative monitoring.

The prototype supports the main report loop. A citizen can register, sign in, create a report, choose category and address fields, add coordinates, mark urgency and severity, and attach photo evidence. Relief users can review wider report lists and map-related screens. Administrators can sign in through the web dashboard, inspect reporter information, search and filter reports, preview evidence, and update report status as pending, verified, resolved, or rejected. For live tracking, connected clients can send location payloads through Socket.IO; the gateway checks latitude and longitude before broadcasting the update.

This research focuses on system design and prototype implementation, not flood prediction or official emergency dispatch. The result is a working reporting pipeline from mobile submission to backend storage and admin review. It also shows how a small microservice design, shared infrastructure code, media storage, role-based access, and container-based deployment can be combined for a Vietnam-focused flood reporting system.

Keywords: VietFlood; flood reporting; relief coordination; real-time tracking; microservice architecture; NestJS; React Native; Next.js; RabbitMQ; PostgreSQL; Redis; Cloudinary.

Chapter 1
Introduction

1.1 The necessity of flood reporting and relief coordination by information technology

1.1.1 What is flood reporting and relief coordination by technology?

Flood reporting begins with a simple fact: someone near the water usually sees the problem before a formal report exists. A resident may take a photo of a flooded road. A volunteer may know that one ward is still passable while another is blocked. A short message can be fast, but it is easy to lose once many people start reporting at the same time.

In this thesis, flood reporting by information technology means turning those field observations into structured records. This approach follows the broader idea that local, citizen-generated observations can become useful geographic and crisis information when they are organized for later processing [1, 2]. A useful report should keep the category, description, province, ward, address line, coordinates, urgency, severity, evidence, reporter, and review status together. Without that structure, a reviewer has to rebuild the situation from scattered messages.

VietFlood is developed for this problem. It is a multi-platform prototype for flood reporting, relief coordination, and live tracking in Vietnam. The system connects a mobile application for citizens and relief workers, a web dashboard for administrators, a NestJS backend, and a shared TypeScript infrastructure package [3, 4]. It does not try to predict floods. It starts later, when people have already seen the water and need to report it in a way that can be checked.

The backend, named `VietFlood`, is divided into an API gateway, an authentication service, a reports service, and a Socket.IO tracking gateway. The API gateway exposes the public HTTP and WebSocket entry points. The authentication service handles registration, sign-in, profile loading, refresh tokens, logout, and user management. The reports service handles flood report creation, listing, update, deletion, status changes, evidence metadata, and Redis cache updates.

The system also uses a shared package named `vietflood_common`. This package contains the logger, Redis module, and Cloudinary helpers used by the backend services. Keeping these utilities in one place matters because the services need consistent logging, cache access, and media handling. If each service copied its own version, small differences would appear quickly.

1.1.2 Some approaches to supporting flood reporting and relief coordination using technology

Several technology choices are used in VietFlood. The first is a mobile-first reporting flow. Flood reports often happen outside, not at a desk. The mobile app is built with Expo React Native and TypeScript [5, 6, 4]. It supports authentication, profile screens,

report creation, image picking, location support, relief views, map-related screens, user overview screens, and role-aware navigation. The app also uses the Vietnam Provinces Open API so users can work with province, district, and ward fields instead of typing every address detail from memory [7].

The second approach is role-based review. VietFlood separates citizen, relief, and admin responsibilities. Citizens create reports and view their own submissions. Relief users can inspect broader report lists and field-oriented screens. Administrators use the web dashboard to search reports, filter by status, inspect evidence thumbnails, read reporter details, update report status, and manage users. The web dashboard blocks non-admin accounts after profile loading. Hiding a page in the client is not enough; the signed-in role still has to be checked.

The third approach is media evidence handling. A flood report is often much clearer when it has a photo. VietFlood does not store raw images inside PostgreSQL. The API gateway receives multipart uploads through the `files` field, uploads each file to Cloudinary under the `vietflood/reports` folder, and passes the returned URL, public ID, and resource type to the reports service [8]. The URL is used for viewing. The public ID is kept so the backend can remove the remote file when a report is deleted.

The fourth approach is a small microservice architecture. Microservice literature describes this style as a way to split a system around independently owned capabilities rather than one large application [9, 10, 11]. In VietFlood, RabbitMQ carries messages between the API gateway, auth service, and reports service [12, 13]. The auth service consumes messages from `auth_queue`; the reports service consumes messages from `reports_queue`. PostgreSQL stores users, refresh tokens, and reports through TypeORM, while Redis keeps cached report lookup data [14, 15, 16]. This setup is heavier than a single backend, but it makes ownership clearer: identity belongs to the auth service, report data belongs to the reports service, and client-facing access belongs to the gateway.

The fifth approach is live location broadcasting. A connected client can emit location data through Socket.IO [17]. The gateway checks that latitude and longitude are valid numbers before broadcasting the update. Invalid payloads return an error event. When a socket disconnects, the gateway notifies connected clients. This is not a full dispatch system. It is a first live channel that fits the prototype.

1.2 Problem Statement

Flood information becomes less useful when it is separated from context. A photo without a location may force a reviewer to ask again. A location without evidence may be hard to verify. A report without a status can sit in a list without anyone knowing whether it has been checked. These are ordinary problems, but during heavy rain they can slow down coordination.

The main problem addressed in this thesis is:

> How can VietFlood collect flood reports from citizens, store the report data and evidence safely, and present the reports in a form that relief users and administrators can review?

This problem has several parts. First, the report form must ask for enough information without making submission too slow. VietFlood focuses on category, description, administrative address, coordinates, urgency, severity, and images. Second, the system must keep evidence connected to the report. Cloudinary stores the files, while PostgreSQL

stores the report record and evidence metadata. Third, reports need a visible review state. VietFlood uses `pending`, `verified`, `resolved`, and `rejected`. Fourth, the same backend must serve both the mobile app and the web dashboard without duplicating business logic in each client.

The prototype also has limits. It does not include flood prediction, satellite imagery analysis, SMS broadcast, official dispatch integration, rescue routing, offline-first synchronization, or formal GIS layers. Those features would need more data, partners, and safety review. VietFlood stays with the part that was built and can be tested: report submission, evidence upload, role-based review, live location events, and container-based deployment.

1.3 Scope And Objectives

The scope of VietFlood covers four project modules. The first module is the backend service group, `VietFlood`. It contains the API gateway, auth service, reports service, and tracking gateway. The gateway exposes authentication and report routes, accepts evidence uploads, and handles Socket.IO events. The auth service manages accounts and tokens. The reports service stores report records, updates statuses, manages evidence metadata, and clears stale Redis cache values when reports change.

The second module is `vietflood_common`. It provides shared backend infrastructure: structured logging, Redis access, and Cloudinary upload or deletion helpers. The logger can include service name, trace context, user ID, role, request path, and method when request context is available.

The third module is `VietFlood_mobile`. It supports citizen reporting and relief workflows through Expo React Native. The mobile app includes login, registration, profile screens, report creation, image attachment, report lists, relief report details, maps, user overview, settings, and Vietnamese or English text. It also normalizes backend data because API payloads may use different field names such as `lat`, `lng`, latitude, longitude, snake case, and camel case.

The fourth module is `Vietflood_web`. It is a Next.js dashboard for administrative monitoring. The dashboard signs in through the backend, stores access and refresh tokens, refreshes sessions when needed, loads the current profile, rejects non-admin users, and displays report cards with search, status filters, evidence preview, reporter information, and status update controls.

The deployment scope includes Docker Compose for the backend services, Redis, and RabbitMQ [18]. The production setup uses Docker Hub images and Azure App Service [19]. GitHub Actions builds the service images, tags them, pushes them to Docker Hub, configures the Azure App Service deployment, and restarts the application [20]. Secrets such as database URLs, RabbitMQ credentials, Redis password, JWT secrets, refresh-token secrets, Cloudinary credentials, and admin seed values are handled through environment configuration.

1.3.1 Goals of VietFlood

The main goal of VietFlood is to build a working flood reporting prototype for Vietnam. The system should let citizens submit reports from a mobile device, preserve evidence, protect review actions by role, and give administrators a readable dashboard for follow-up.

The specific goals are:

1. Design a NestJS backend with an API gateway, authentication service, reports service, and Socket.IO tracking gateway.

2. Use RabbitMQ so the gateway can communicate with internal services through message patterns.

3. Store users, refresh tokens, and flood reports in PostgreSQL through TypeORM.

4. Use JWT access tokens, refresh tokens, and role guards for citizen, relief, and admin workflows.

5. Allow citizens to create reports with category, description, address fields, coordinates, urgency, severity, and photo evidence.

6. Upload evidence to Cloudinary and store the returned URL and public ID with the report.

7. Use Redis for report lookup caching and clear stale cache entries when reports change.

8. Build an Expo React Native mobile app for citizen and relief workflows.

9. Build a Next.js web dashboard for admin login, report search, evidence review, user management, and status updates.

10. Prepare Docker Compose and CI/CD support for local and production-style backend deployment.

1.3.2 System Actors and Basic Functions

The VietFlood prototype uses three main roles: citizen, relief, and admin. The roles are intentionally small. A real disaster-response organization may need more titles, but this prototype only needs enough separation to protect report ownership and reviewer functions.

Citizens use the mobile application to register, sign in, manage their profile, create flood reports, attach evidence, provide location information, and view their own reports. This role is designed for people who are close to the incident and need a quick way to submit information.

Relief users need broader report access. In the mobile app, relief-oriented screens allow these users to inspect report lists, open report details, work with map-related views, and update report status where allowed. This role supports field review rather than full system administration.

Administrators use the web dashboard. They can sign in, load all reports, search by report or reporter fields, filter by status, preview evidence thumbnails, inspect reporter details, update report status, and manage user records. Admin access is checked after profile loading, so a non-admin account cannot use the dashboard simply by opening the page.

These actors share one backend entry point. Mobile and web clients call the API gateway. The gateway decides whether a request should become an auth message, a report message, an evidence upload, or a Socket.IO event. This keeps the client applications simpler and keeps service responsibilities visible.

1.3.3 Thesis Structure

This thesis is organized as follows. Chapter 1 introduces the motivation, problem, scope, objectives, actors, and structure of the VietFlood project. Chapter 2 reviews background and related work for flood reporting, role-based review, microservice architecture, message queues, evidence storage, caching, real-time communication, and deployment.

Chapter 3 presents the materials and methods used to build VietFlood, including the project modules, actors, architecture, data design, workflows, shared package, deployment method, and evaluation plan. Chapter 4 describes the implementation and result of the backend, mobile application, web dashboard, shared library, and deployment setup. Chapter 5 discusses the project results, limitations, conclusion, and possible future work.

Chapter 2
Background and Related Works

2.1 Background

Flood response begins with information that is usually incomplete. A resident may know that a road is under water, but not the official ward name. A volunteer may receive a photo through a chat group, but the message may not include coordinates. A local administrator may hear about a blocked street, yet still needs to know whether the case has already been checked by somebody else. VietFlood was designed around this practical problem. It is a reporting and coordination prototype that turns citizen observations into structured records that can be reviewed by relief workers and administrators, following the same general need for organized volunteered information in crisis response [1, 2].

The project does not try to replace weather forecasting systems or official emergency command centers. Its focus is narrower. VietFlood collects field reports through a mobile application, stores each report with location and evidence, allows reviewers to update the report status, and gives administrators a web dashboard for monitoring. This scope is suitable for a graduation project because it contains real engineering problems without pretending to solve every part of disaster management.

2.1.1 Flood Reports as Structured Field Records

In many flood situations, the first useful record is a message from somebody near the incident. The message might be a short description, a photo, a location pin, or a call for help. That kind of information is fast, but it is difficult to search and verify when it stays inside a chat thread. VietFlood treats each citizen submission as a field record.

The report model used in VietFlood stores the incident category, written description, image links, evidence metadata, province, ward, address line, latitude, longitude, severity, urgency flag, status, timestamp, and the account that created the report. These fields are not decorative. They answer the questions that a reviewer usually asks first: what happened, where it happened, how serious it seems, who submitted it, and what evidence came with it.

The report status is deliberately small. A report can be `pending`, `verified`, `resolved`, or `rejected`. This is enough for the prototype. A new report starts as pending. A reviewer can mark it as verified when the information looks trustworthy. A resolved report means the case has been handled or is no longer active. A rejected report is kept separate from verified work so that incorrect or unsuitable reports do not disappear without a trace.

2.1.2 The Role of Mobile Reporting

The mobile application is important because most flood observations happen outside an office. A user may be standing near a flooded street, using mobile data, and trying to submit the report quickly. VietFlood therefore uses an Expo React Native application for the citizen and relief-side experience [5, 6].

Figure 2.1 shows the basic life of a citizen report. The flow starts with a mobile submission, then passes through the gateway, evidence storage, report creation, reviewer

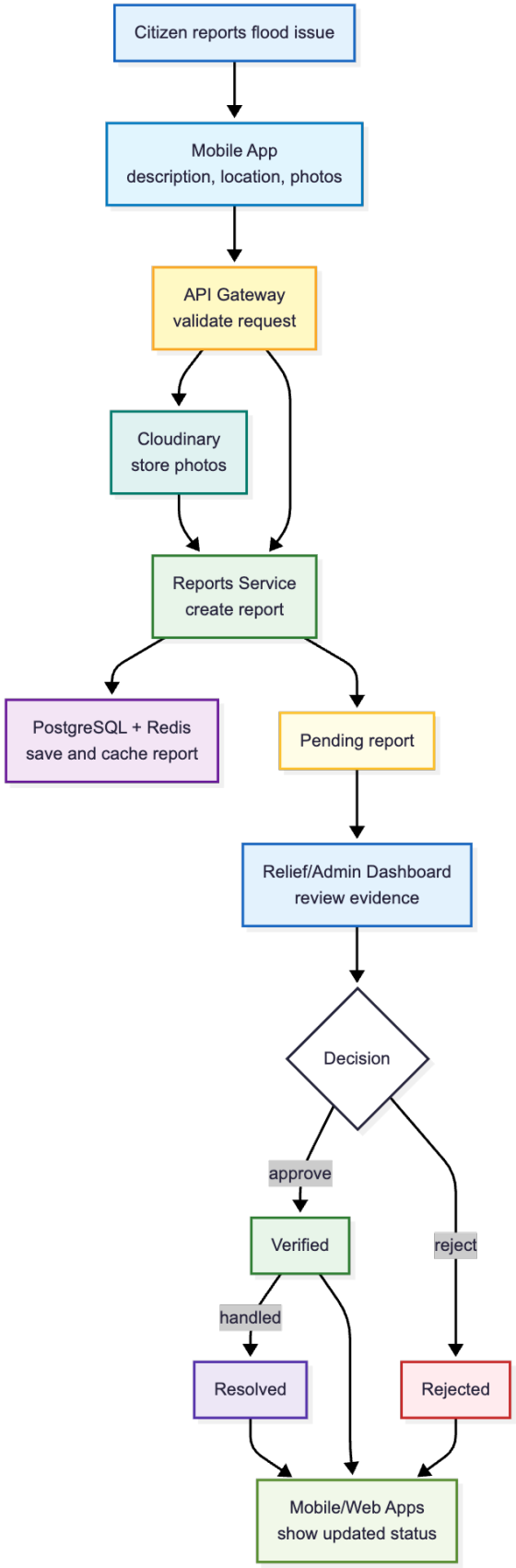

Figure 2.1: Lifecycle of a VietFlood citizen report from mobile submission to review status.

inspection, and status update. The diagram stays simple because this section focuses on the mobile reporting role. Later chapters can show the detailed backend architecture.

The mobile app includes registration, login, profile screens, report creation, image selection, report lists, relief screens, map views, and role-aware navigation. It also works with the Vietnam Provinces Open API so that province, district, and ward data can be selected from known administrative divisions instead of being typed freely every time [7]. This reduces spelling variation in location data. It also makes later filtering easier.

Location is handled as a first-class part of the report. When permission is available, the mobile app can use device location to help fill the position of a report. The backend stores latitude and longitude as report fields. The mobile client also normalizes data when backend and frontend names are slightly different, such as `lat`/`lng` and `latitude`/`longitude`. That kind of defensive handling is ordinary in real client applications. It prevents one naming mismatch from breaking a whole screen.

2.1.3 The Role of Web-Based Review

The web dashboard has a different audience. It is not built for a citizen standing in the rain. It is built for an administrator who needs to scan many reports, check evidence, filter records, and update statuses. VietFlood's web client is a Next.js application using React, TypeScript, and Tailwind CSS [21, 22, 4, 23]. It works as an admin dashboard called VietFlood Insight.

The dashboard loads report data through the API gateway. It displays status, category, severity, address, description, evidence thumbnails, reporter information, and creation time. It also supports searching and filtering by report fields, reporter fields, and status. This matters because a reviewer may look for reports in one ward, reports submitted by one person, or all reports still marked as pending.

The dashboard is protected by authentication and role checking. A normal citizen account should not be able to use the administrator workspace. The web client validates the signed-in profile and blocks non-admin users, while the backend protects sensitive routes with JWT and role guards. Client-side hiding is convenient for the interface, but server-side protection is the part that enforces the rule.

2.2 Related Work and Technical Foundations

The related work for VietFlood is not one single application that the project copies. It comes from several practical patterns: mobile field reporting, role-based review systems, microservice backend design, message queues, external media storage, caching, real-time socket communication, and containerized deployment. This section explains the patterns that shaped the project.

2.2.1 Informal Reporting Channels

In many communities, reports about flooding first appear in phone calls, messaging apps, social media posts, or local volunteer groups. These channels are useful because they are quick and familiar. People already know how to send a photo or forward a location pin. During a flood, speed matters.

The weakness is that informal channels are hard to organize later. A chat group can contain duplicate reports, missing addresses, old photos, and messages from people with different levels of reliability. It is also difficult to assign a status to each case. A message

may be read by one volunteer but invisible to another. A photo may be useful, but it can be separated from the original location or reporter.

VietFlood keeps the speed of citizen reporting while adding structure. The mobile form asks for the incident type, description, address information, location, and evidence. The backend stores this as a searchable report record. The web dashboard then gives reviewers a shared place to inspect and update that record.

2.2.2 Centralized Web Systems

A common first design for this type of application is a centralized web system. One server handles login, reports, files, users, and the admin interface. This design is easy to understand. It also makes local development simple because all application logic is in one process.

For a small prototype, a monolithic backend can be a reasonable choice. VietFlood, however, uses a small microservice structure because the project needed to practice service boundaries, queue-based communication, and independent backend modules. This follows the architectural argument that services should be separated where boundaries make responsibilities clearer, while still considering the added operational cost [9, 11, 10, 24]. The split is intentionally modest. The backend is not divided into dozens of services. It has an API gateway, an authentication service, a reports service, Redis, RabbitMQ, PostgreSQL, and Cloudinary integration.

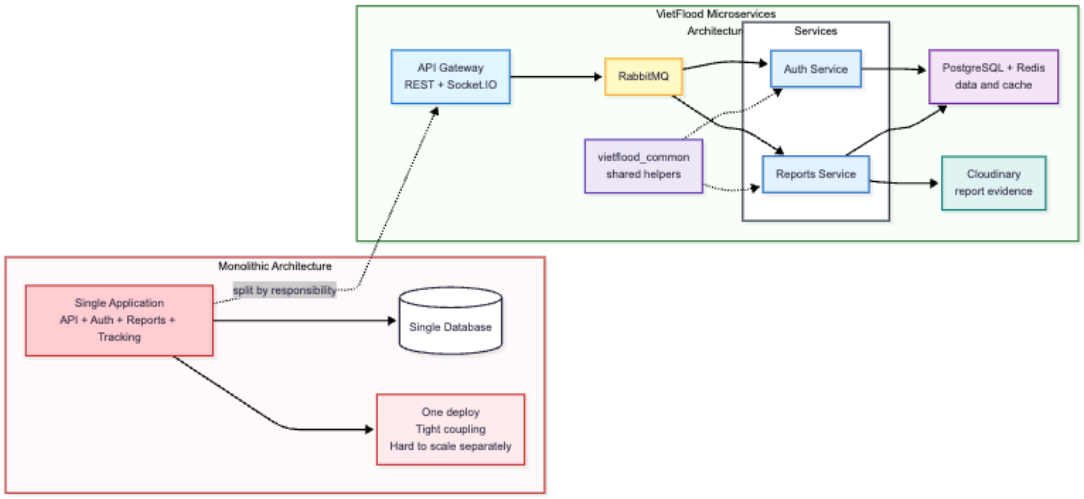

Figure 2.2: Comparison between a monolithic backend and the VietFlood microservices structure.

Figure 2.2 can be used to show why the project separates responsibilities. The API gateway handles public HTTP requests and Socket.IO events. The auth service handles accounts, passwords, access tokens, refresh tokens, and profiles. The reports service handles report records, evidence metadata, status changes, cache updates, and report deletion. This split keeps each service closer to one responsibility.

2.2.3 API Gateway Pattern

The API gateway is the public entrance to the backend. Mobile and web clients do not call the auth service or reports service directly. They call the gateway. This design follows a common architectural pattern where a public-facing component shields clients from internal service structure [24, 3]. The gateway receives HTTP requests, validates

authentication where needed, accepts uploaded evidence files, and forwards service-specific work to the correct backend service.

This pattern gives the clients one stable backend address. It also keeps internal service names and queues away from the frontend. In VietFlood, the gateway exposes routes for authentication, reports, tracking, and user-related actions. For report creation, it receives multipart files under the `files` field, uploads the file buffers to Cloudinary, and sends the final payload to the reports service.

The gateway also hosts Socket.IO tracking. It listens for a `send-location` event, checks that latitude and longitude are valid numbers within the correct range, and broadcasts a `receive-location` event when the payload is valid. If the payload is not valid, the client receives `location-error`. When a socket disconnects, the gateway emits `user-disconnected`. The feature is still a prototype, but it shows where live movement data can enter the system.

2.2.4 Message Queues Between Services

The VietFlood backend uses RabbitMQ for communication between the gateway and backend services [12, 13, 25]. Instead of importing service logic directly, the gateway sends message patterns to queues. Authentication messages go to the auth queue. Report messages go to the reports queue. The services consume those messages and return results.

This design makes the boundary between services visible. A login request, for example, is handled by the gateway at the HTTP layer, but the password check and token creation belong to the auth service. A report creation request is received by the gateway, but report persistence belongs to the reports service. The gateway is a router and coordinator, not the owner of every business rule.

The project also uses timeouts and retries around service calls. This is important because distributed systems fail in smaller pieces. If a service becomes slow, the gateway should not leave the client waiting forever. A controlled error is easier to understand than a frozen request.

2.2.5 Authentication and Role-Based Access

VietFlood uses username and password authentication. During registration, the auth service hashes passwords with bcrypt before saving them [26]. During sign-in, it checks the password and returns an access token and refresh token. The access token follows the JWT style of carrying signed claims for later authenticated requests [27]. The access token includes the user ID, username, role, first name, and last name.

The role model separates citizen, relief, and admin behavior. A citizen creates reports and views personal report history. A relief user can review broader report information and work with relief-oriented screens. An admin can use the web dashboard, inspect reports, update report status, and manage user information. The exact role names are simple, but the separation is necessary. Without it, every authenticated user would have too much power.

The mobile app uses role-aware navigation so that each user lands in a suitable workspace. The web app checks for an admin profile before showing the dashboard. The API gateway and services enforce protected routes on the backend side. This gives the system both usability and access control.

2.2.6 Media Evidence and Cloud Storage

Photo evidence makes a report more useful. A short description may say that a street is flooded, but a photo can help a reviewer judge the water level, blocked access, damaged infrastructure, or possible danger. VietFlood stores media files outside the database by using Cloudinary [8].

The gateway receives uploaded files as buffers. It sends them to Cloudinary under the `vietflood/reports` folder. After upload, it passes structured evidence data to the reports service. The report entity stores evidence items with a URL, public ID, and resource type. It also keeps image URLs for screens that only need preview images.

The public ID is important because report deletion should also remove the remote asset. A display URL alone is not enough to manage media lifecycle. The shared `vietflood_common` package provides Cloudinary helper functions so that services do not each implement their own upload and delete logic.

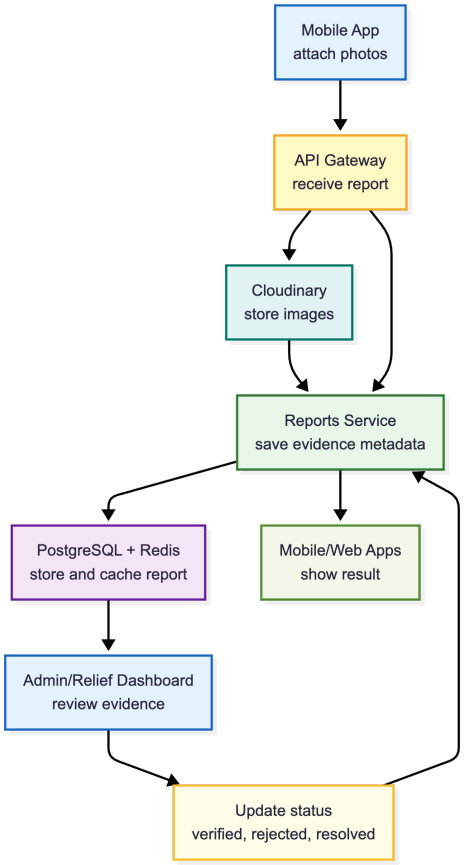

Figure 2.3: Evidence upload and review flow in VietFlood.

Figure 2.3 explains the system path, but the thesis should also include real screenshots from the application. The mobile screenshot should show a citizen attaching photo evidence while creating a report. The dashboard screenshot should show how a relief worker or administrator reviews the submitted evidence before changing the report status.

2.2.7 Database and Cache

PostgreSQL is the main database for VietFlood. It stores users, refresh tokens, reports, and other persistent records. The backend uses TypeORM entities to map application objects to database tables [14]. This gives the services a consistent way to work with

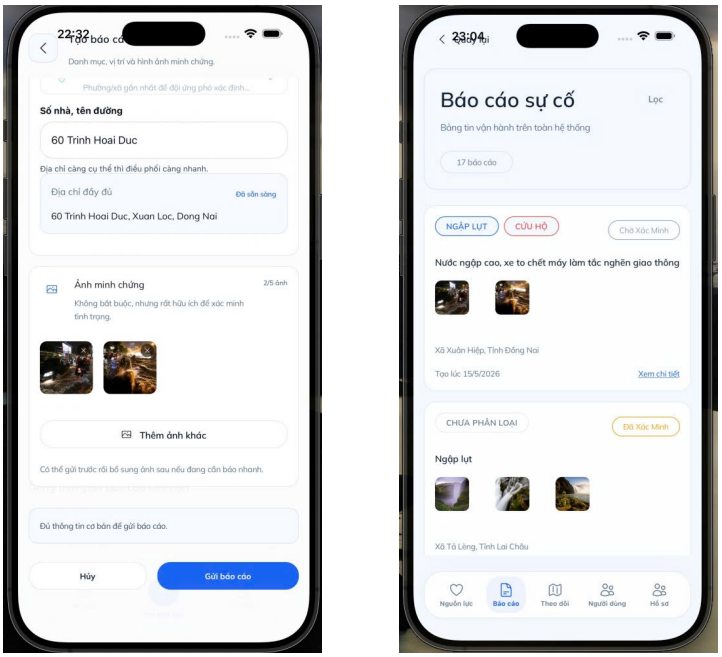

Figure 2.4: Mobile evidence upload and relief dashboard evidence review in VietFlood.

users and reports while still relying on a relational database.

Redis is used as a cache [16]. The reports service stores report entries with a key pattern such as `report:id`. When a report is fetched, the service can check Redis before reading from PostgreSQL. When a report is changed or deleted, the cache entry is refreshed or removed. This pattern keeps PostgreSQL as the source of truth while using Redis to reduce repeated reads.

The cache design is intentionally narrow. It would be risky to cache everything too early. In VietFlood, caching is used where repeated report lookup is useful and where invalidation can be kept understandable. That is a better fit for the prototype than a broad cache layer that hides stale data.

2.2.8 Shared Infrastructure Package

The backend services reuse common infrastructure through the `vietflood_common` package. This package exports Redis helpers, Cloudinary helpers, and a structured logger. The logger can include timestamp, level, service name, trace context, user ID, role, path, and request method when that context is available.

This shared package reduces repeated code across the gateway, auth service, and reports service. It also keeps infrastructure behavior consistent. For example, a Cloudinary upload should not behave differently just because it was called from another service. A Redis connection should follow the same configuration approach. A log line should have enough context to trace a request through the backend.

2.2.9 Containerized Deployment

VietFlood uses Docker Compose to run backend services with Redis and RabbitMQ [18]. The local setup builds the API gateway, auth service, and reports service from the repository. The production setup points to Docker Hub images. Environment variables provide database URLs, Redis settings, RabbitMQ credentials, JWT secrets, refresh token secrets, Cloudinary credentials, and admin seed values.

The project also includes a GitHub Actions workflow. It builds Docker images for the gateway, auth service, and reports service, pushes them to Docker Hub, and deploys the multi-container configuration to Azure App Service [20, 19]. This does not remove the need for careful configuration, but it gives the project a repeatable release path, which is consistent with continuous delivery and DevOps practices that emphasize repeatable build and deployment pipelines [28, 29].

2.3 Comparison of Design Approaches

The project can be understood through three possible approaches. The first approach is informal reporting through chat and social media. It is fast and familiar, but it does not give the team a structured report database. The second approach is a single-server web application. It is simpler to build and debug, but it gives less practice with service separation and can become difficult to maintain when authentication, reporting, media handling, and live tracking grow together. The third approach is the VietFlood architecture: a mobile app and web dashboard connected to a gateway, with dedicated services for authentication and reports.

The VietFlood approach costs more engineering effort. RabbitMQ, Docker Compose, service messages, environment variables, and shared packages all add work. The benefit is that the main responsibilities are clearer. The gateway owns the client-facing layer. The auth service owns identity. The reports service owns flood reports. Redis and Cloudinary support specific infrastructure needs instead of being hidden inside unrelated code.

This design is suitable for the thesis because it shows a realistic tradeoff. The system is not overbuilt into many small services. It is also not limited to a single backend file. It sits in the middle, where the project can demonstrate mobile reporting, admin review, service communication, evidence storage, caching, live events, and deployment practice.

2.4 Chapter Summary

This chapter explained the background and related technical ideas behind VietFlood. Flood reports are treated as structured field records rather than loose messages. The mobile application supports fast citizen reporting, while the web dashboard supports administrator review. The backend uses an API gateway, RabbitMQ, dedicated services, PostgreSQL, Redis, Cloudinary, and a shared infrastructure package.

The next chapter can build on this foundation by describing the materials and methods in more detail: system requirements, actors, architecture, database design, service workflows, development tools, and testing approach.

Chapter 3
Methodology

This chapter explains the method used to design and build VietFlood. The work followed a practical software-engineering path: identify the users, define the report data that must survive across the system, split the application into services, implement the mobile and web clients, then test the main workflows against the deployed backend. This method follows the idea that architecture should be evaluated through the responsibilities and qualities required by the system, not only through technology choice [24]. The method is not based on a public survey like the previous template chapter. VietFlood was built from the project proposal, the source code, and the repeated workflow of creating, reviewing, and updating flood reports.

3.1 Requirement Analysis

The first step was to reduce the problem to a set of users and actions that could be implemented in a graduation project. Flood response is a large topic, but VietFlood focuses on one narrow part of it: a citizen submits a field report, relief staff or administrators inspect the report, and the system keeps the report status visible to the clients.

3.1.1 Functional Requirements

The functional requirements were grouped around the three client-facing roles in the system.

- **Citizen users** can register, sign in, update their profile, create flood reports, attach photo evidence, provide address information, and view their own reports.
- **Relief users** can view broader report lists, open report details, inspect location and evidence, and coordinate field response from the available report information.
- **Admin users** can use the web dashboard to view all reports, search and filter records, inspect reporter information, preview evidence, update report status, and manage user information.

The report itself was treated as the central object of the system. Each report needed to store category, description, address, latitude, longitude, evidence, reporter, severity, urgency, and status. The status values were kept short: `pending`, `verified`, `resolved`, and `rejected`. A small status model is easier to test and easier for users to understand.

3.1.2 Non-Functional Requirements

The non-functional requirements came from the nature of the workflow. The backend should accept reports from both mobile and web clients through one public entry point. Services should be separated enough that authentication and reports do not become one large module. Uploaded evidence should be stored outside the database. The system should be deployable with repeatable commands rather than manual server setup.

These requirements led to four design decisions. First, the backend uses an API gateway in front of separate auth and reports services. Second, RabbitMQ is used for service communication. Third, Cloudinary stores report evidence files while PostgreSQL stores report data and evidence metadata. Fourth, Docker Compose is used to run the backend services and infrastructure in a repeatable environment.

3.2 Actor Analysis

VietFlood uses three main actors. The old template chapter used separate use-case figures for visitor, author, and administrator actors. In VietFlood, those figures are replaced by citizen, relief, and administrator use cases.

3.2.1 Citizen Actor

The citizen actor represents the person who observes a flood-related incident and submits the first record. The citizen workflow begins with authentication, then moves to report creation. The mobile app collects the category, description, location fields, and optional photos. It also supports device-location assistance and Vietnam administrative division selection.

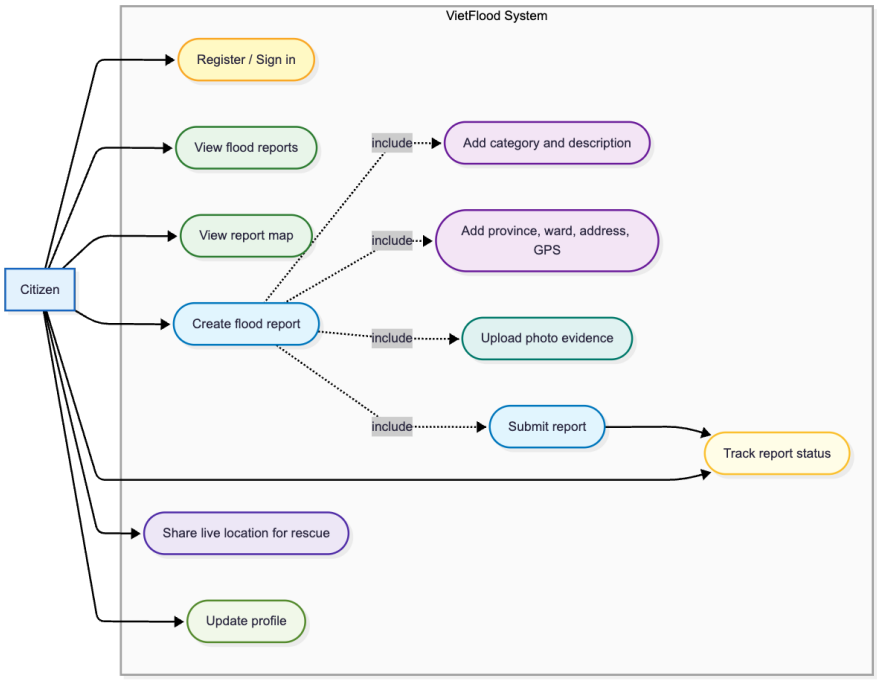

Figure 3.1: Use case diagram for the citizen actor in VietFlood.

3.2.2 Relief Actor

The relief actor represents staff who need to review field information. In the mobile app, relief-oriented screens include report lists, report details, relief maps, and location-oriented workflows. The relief user does not create the original citizen report in the normal case, and this role does not verify or reject submitted reports. The main job is to read what has been submitted, inspect the location and evidence, and coordinate response work from the available information.

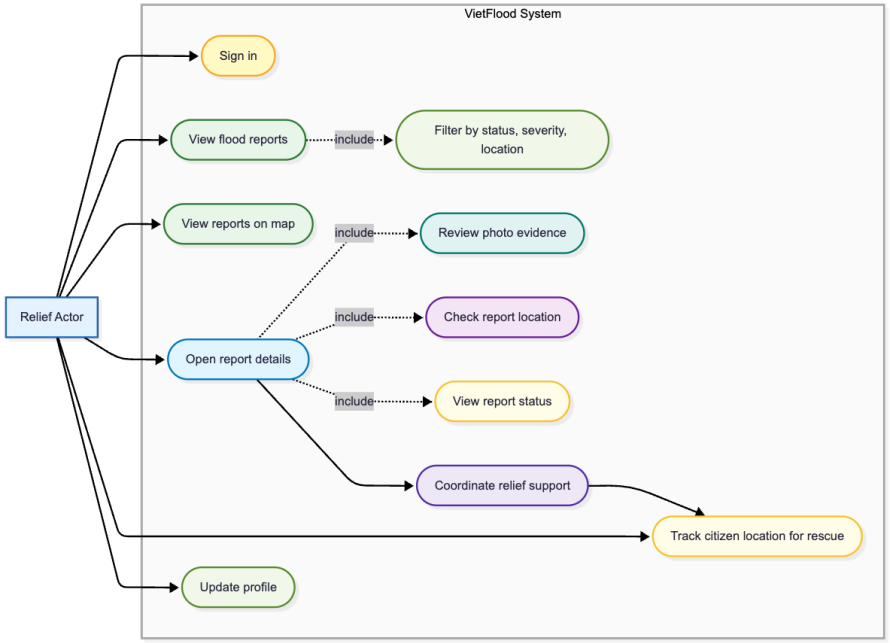

Figure 3.2: Use case diagram for the relief actor in VietFlood.

3.2.3 Administrator Actor

The administrator actor works mainly through the web dashboard. The dashboard loads report data from the API gateway, combines report information with reporter data, and provides search, filtering, evidence previews, and status update controls. The web client also checks that the signed-in profile is an admin before showing the dashboard.

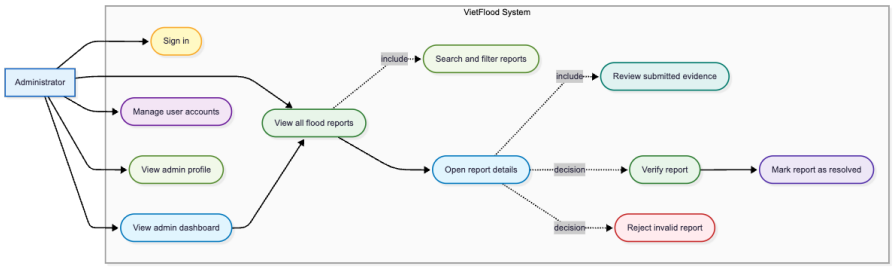

Figure 3.3: Use case diagram for the administrator actor in VietFlood.

3.3 Materials and Development Tools

The project was built from four source-code modules. The backend module contains the NestJS API gateway and microservices [3, 12]. The shared common module provides reusable logging, Redis, and Cloudinary helpers. The mobile module contains the Expo React Native application [5, 6]. The web module contains the Next.js dashboard [21].

3.4 System Design Method

The backend was designed as a small microservice system. The service split was kept small because microservices are useful only when their boundaries help the design instead of adding unnecessary operational weight [9, 11, 10]. The API gateway is the public entry

Table 3.1: Main materials and tools used in VietFlood development.

Area	**Tools and purpose**
Backend	NestJS, TypeScript, TypeORM, PostgreSQL, RabbitMQ, Redis, Socket.IO, JWT, bcrypt.
Shared package	`vietflood_common` for structured logging, Redis client setup, and Cloudinary file operations.
Mobile client	Expo, React Native, TypeScript, React Navigation, Redux Toolkit, image picking, secure token storage, device location, and Vietnam Provinces Open API.
Web dashboard	Next.js, React, TypeScript, Tailwind CSS, protected admin login, token refresh, report overview, search, filtering, evidence preview, and status update.
Runtime infrastructure	Docker, Docker Compose, PostgreSQL, Redis, RabbitMQ, Cloudinary, and Azure App Service configuration.

point. It receives HTTP requests from the mobile and web clients and forwards work to the correct service. The auth service owns registration, sign-in, refresh tokens, profile loading, and user management. The reports service owns flood report creation, report queries, evidence metadata, status updates, cache updates, and report deletion.

This split was chosen because authentication and reporting have different rules and apply different design concerns. A login request should not be mixed with evidence upload logic. A report status update should not be stored inside the web dashboard. Following established design patterns for separation of concerns and single responsibility [30, 24], each service keeps a clear responsibility, and the gateway coordinates the request flow.

3.4.1 API Gateway Method

The gateway exposes the main client-facing routes. Authentication routes are grouped under `/auth`. Report routes are grouped under `/reports`. Report creation uses multipart upload through a field named `files`. The gateway stores uploaded file buffers in memory long enough to send them to Cloudinary. After upload, it sends the report payload and evidence metadata to the reports service. This gateway-centered design matches the NestJS approach used by the backend applications [3, 12].

The gateway also applies JWT guards and role guards. Citizens can reach citizen routes. Relief and admin users can reach broader report-review routes. Admin-specific update routes are protected separately. This keeps the client interface simple while still enforcing access rules on the backend.

3.4.2 Message Communication Method

RabbitMQ is used between the gateway and services [13]. The auth service listens for message patterns such as `register`, `sign_in`, `profile`, `refresh_token`, and `logout`. The reports service listens for `create`, `get_all_by_users`, `update`, and `delete`. Gateway calls use timeout and retry behavior so a slow service does not leave the client waiting without a response.

This message-based method also made the service boundary easier to see during de-

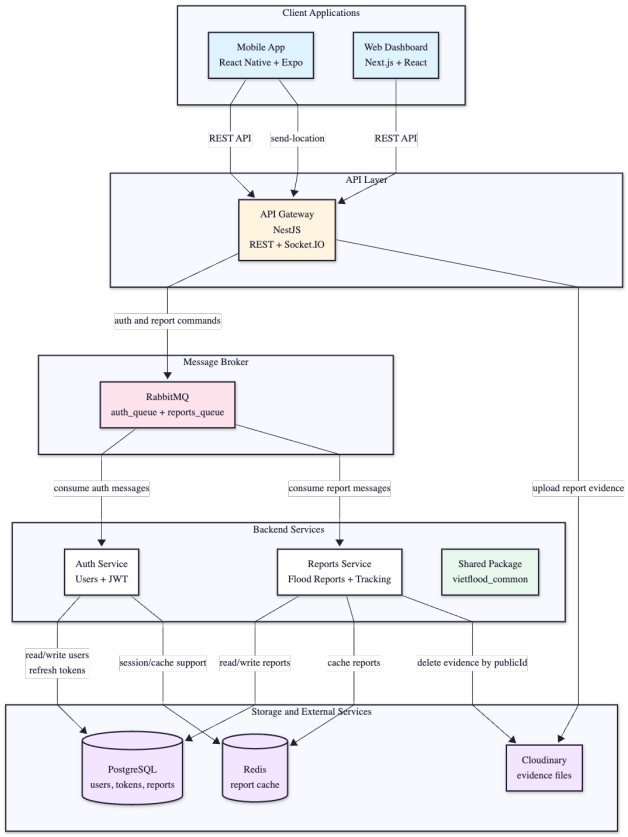

Figure 3.4: VietFlood system architecture diagram.

velopment. If report creation failed, the failure could be checked at three points: the gateway request, the Cloudinary upload, and the reports service message handler.

3.5 Database Design Method

The database design started from the report workflow. A report needs a user owner, a location, a status, and evidence. The backend uses PostgreSQL as the main data store and TypeORM entities for users, refresh tokens, and reports [14, 31].

The user entity stores email, username, phone, hashed password, role, name fields, date of birth, address fields, and timestamps. Email, username, and phone are unique. Roles are stored as `citizen`, `relief`, or `admin`.

The report entity stores category arrays, description, image URLs, evidence JSON, province, ward, address line, latitude, longitude, status, reporter name, urgency, severity, timestamps, and owner user ID. Evidence is stored as JSON because each item needs a URL, Cloudinary public ID, and resource type, and PostgreSQL supports JSON-style storage for this kind of flexible metadata [15]. The public ID is used later when a report is deleted and the related media must also be removed.

3.5.1 Cache Method

Redis is used as a cache rather than as the source of truth [16]. When a report is created, the reports service stores a cached entry using the `report:id` pattern. When a report is requested by ID, the service checks Redis before querying PostgreSQL. When a report is updated or deleted, the cache entry is removed or refreshed.

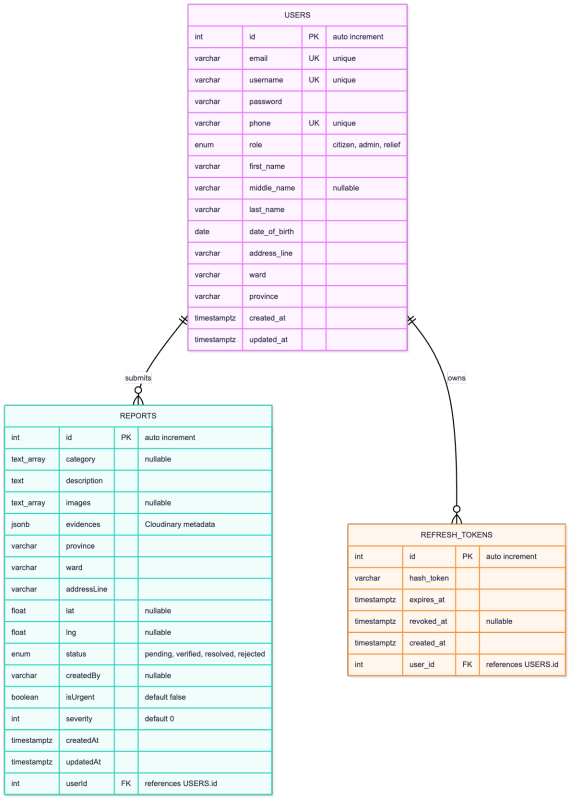

Figure 3.5: Database schema diagram for VietFlood.

The cache method is narrow on purpose. Report ownership, evidence, status, and location are still stored in PostgreSQL. Redis only keeps a temporary copy for faster lookup.

3.6 Workflow Design Method

The main workflow in VietFlood is report creation. The citizen selects categories, writes a description, chooses location fields, and attaches optional images. The mobile app builds a multipart form. The API gateway validates the authenticated user, uploads file buffers to Cloudinary, and sends the payload to the reports service through RabbitMQ. The reports service saves the record in PostgreSQL and writes the cache entry in Redis.

3.6.1 Evidence Handling Method

Evidence images are not stored inside PostgreSQL as raw files. The gateway uploads each image buffer to Cloudinary under the report folder [8]. The reports service stores the returned URL, public ID, and resource type. The URL is used by the mobile app and web dashboard for display. The public ID is kept so that the system can delete remote files when a report is removed.

This method keeps the database small and gives both clients stable media URLs. It also creates one extra responsibility: Cloudinary credentials must be handled through environment configuration, not source code.

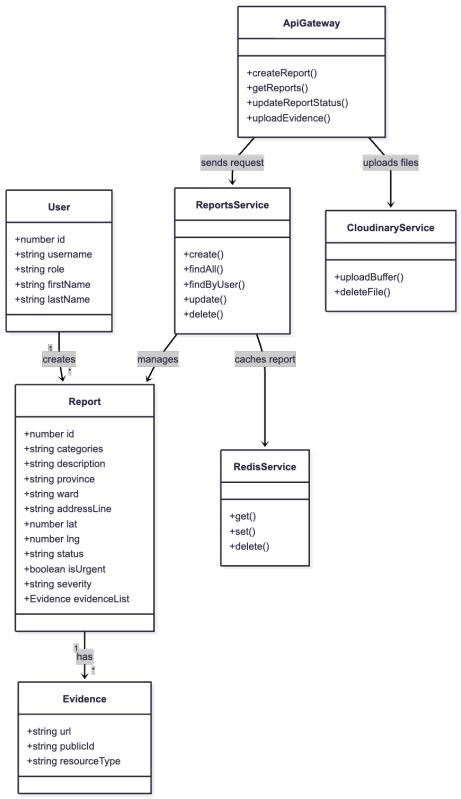

Figure 3.6: Class or module diagram for the report workflow.

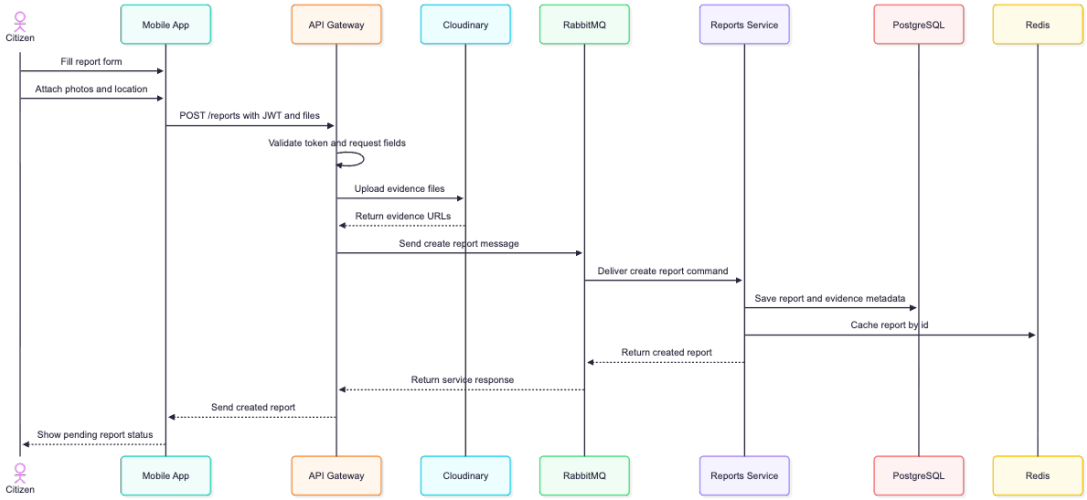

Figure 3.7: Sequence diagram for creating a VietFlood report.

3.6.2 Status Review Method

Status review is handled through the administrator flow. The relief screens display reports with evidence and location information so response staff can understand the field situation, but report verification and rejection are not part of the relief role. The web dashboard sends admin status updates through the gateway route that includes the report ID and owner user ID. The reports service checks the existing report, merges incoming evidence when needed, updates the status, and refreshes the cache.

3.7 Client Application Method

3.7.1 Mobile Application Method

The mobile app was developed with Expo React Native and TypeScript [5, 6, 4]. It uses token-based authentication, profile screens, report creation, report lists, relief report detail screens, map screens, and role-aware navigation. Redux Toolkit supports application state around authentication and client-side data flow [32]. The report service in the mobile app normalizes backend data before rendering it. This was necessary because some fields appear in snake case and some in camel case. The mobile app also accepts location values as `lat/lng` or `latitude/longitude`.

The mobile location method uses two inputs. The first input is administrative division data from the Vietnam Provinces Open API [7]. The second input is device location when permission is available. The app can prefill location fields so the user does not have to type every address value manually.

3.7.2 Web Dashboard Method

The web dashboard was developed with Next.js, React, TypeScript, and Tailwind CSS [21, 22, 4, 23]. It is admin-focused. It loads all reports from the gateway, joins report information with reporter data returned by the backend, counts reports by status, and provides search and filtering. Evidence is displayed through thumbnails when report images are available.

The web client stores access and refresh tokens in browser storage and refreshes access tokens through the backend when needed. The dashboard checks the current profile and blocks non-admin users from the admin workspace.

3.8 Real-Time Tracking Method

The backend includes a Socket.IO gateway for live location updates [17]. A connected client can emit `send-location` with latitude, longitude, accuracy, heading, speed, and timestamp. The gateway validates the latitude and longitude ranges. If the payload is invalid, it emits `location-error`. If it is valid, it broadcasts `receive-location`. When a socket disconnects, the gateway emits `user-disconnected`.

This feature is treated as a prototype feature in the methodology. It proves that the backend can receive and broadcast location events, but future work is needed for authenticated socket sessions, route assignment, and persistent location history.

3.9 Validation Method

Validation was done through source-level checks and workflow checks. The source-level checks focused on whether the services matched their responsibilities: auth handled identity, reports handled report data, the gateway handled client-facing access, and the shared package handled common infrastructure. Docker Compose was used as the repeatable local environment for checking the backend services together with their infrastructure dependencies [18]. Workflow checks focused on the main user paths:

- citizen registration, sign-in, and profile loading;
- citizen report creation with category, address, and images;
- report evidence upload to Cloudinary and metadata storage in PostgreSQL;
- report list loading for citizen, relief, and admin views;
- status update from the admin dashboard;
- Redis cache creation, refresh, and deletion during report changes;
- Docker Compose startup for backend services and local infrastructure.

This validation method is suitable for the current project stage. VietFlood is a working prototype, not a certified emergency-response platform. The tests and checks therefore focus on proving that the main reporting, evidence, review, and service-integration flows work as designed.

Chapter 4
Implementation and Result

This chapter describes how VietFlood was implemented across the backend services, shared common package, mobile application, and web dashboard. The chapter keeps the same implementation-oriented structure as the original template: it begins with the tools used in the project, then explains the service communication layer, authentication flow, report processing, shared infrastructure, mobile features, web dashboard features, deployment flow, and user management.

The implementation is written around the code that exists in the VietFlood repositories. Some figures are left as placeholders with VietFlood file names. These placeholders can be replaced later with screenshots, diagrams, or exported Mermaid images without changing the surrounding text.

4.1 Technique and Tools

VietFlood was implemented as a multi-platform system. The backend uses NestJS and TypeScript [3, 4]. The mobile app uses Expo React Native [5, 6]. The web dashboard uses Next.js [21]. The shared package keeps Redis, Cloudinary, and logging helpers in one place so the gateway and microservices do not duplicate infrastructure code.

a) Version Control and Build Tools:

The project uses Git for source control. The backend is arranged as a NestJS workspace with separate applications for the API gateway, authentication service, and reports service. Each backend service has its own Dockerfile. Docker Compose is used to run the services together with RabbitMQ and Redis [18]. The production workflow builds Docker images for the gateway, auth service, and reports service before deployment to Azure App Service [19].

b) Front-end Development:

The mobile client was implemented with Expo, React Native, and TypeScript [5, 6, 4]. It uses React Navigation for screen movement, Redux Toolkit for authentication state [32], Expo SecureStore for durable token storage, device location APIs for report location support, and image picking for evidence upload. The code also has a session-only token mode, which helps when the user chooses not to keep the login after closing the app.

The web dashboard was implemented with Next.js, React, TypeScript, and Tailwind CSS [21, 22, 4, 23]. The dashboard is admin-focused. It reads reports from the backend, formats report status values, shows evidence thumbnails, filters reports by status, and sends status updates through the API gateway.

c) Back-end Development:

The backend uses NestJS microservices [12]. The API gateway exposes REST routes and the Socket.IO gateway [17]. The auth service handles registration, sign-in, profile loading, profile update, refresh tokens, logout, and user administration. The reports service handles report creation, report retrieval, report updates, report deletion, evidence metadata, and Redis cache updates.

RabbitMQ is used for gateway-to-service messages [13]. The gateway sends authentication work to the auth queue and report work to the reports queue. Each request uses timeout and retry behavior so a service failure can return a controlled error instead of leaving the client waiting indefinitely.

d) Database and Media Storage:

PostgreSQL stores users, refresh tokens, and reports. TypeORM maps these tables into the backend services [14]. Redis is used for report caching with keys such as `report:id` [16]. Cloudinary stores uploaded report evidence [8]. PostgreSQL only keeps the evidence metadata as JSON-style data: public URL, Cloudinary public ID, and resource type [15].

e) Cloud Service and Runtime Configuration:

The deployed backend uses Azure App Service as the runtime target. Configuration is provided through environment variables, including database URL, Redis settings, RabbitMQ credentials, JWT secrets, refresh-token secret, Cloudinary credentials, and admin seed values. This keeps credentials out of the source code and lets the same code run in local and deployed environments.

Table 4.1: Main implementation tools used in VietFlood.

Area	**Tools and implementation use**
Backend	NestJS, TypeScript, TypeORM, RabbitMQ, Socket.IO, JWT, bcrypt, Docker.
Shared library	`vietflood_common` for logging, Redis, and Cloudinary.
Database and cache	PostgreSQL for durable data, Redis for report cache entries.
Media storage	Cloudinary for report evidence files and optimized media URLs.
Mobile app	Expo, React Native, React Navigation, Redux Toolkit, SecureStore, image picker, device location.
Web dashboard	Next.js, React, TypeScript, Tailwind CSS, browser token storage, admin report review.
Deployment	Docker Compose, Docker Hub, GitHub Actions, Azure App Service.

4.2 Messaging Broker Implementation

VietFlood uses RabbitMQ as the message broker between the API gateway and the backend services [13, 12]. The API gateway is the only public HTTP entry point. It receives the client request, prepares the payload, and sends a message to the correct service. The auth service listens on `auth_queue`. The reports service listens on `reports_queue`.

The implementation uses message patterns instead of direct service imports. The auth service handles patterns such as `register`, `sign_in`, `profile`, `refresh`, `refresh_token`, and `logout`. The reports service handles patterns such as `create`, `get_all_by_users`, `update`, and `delete`. This design keeps report logic out of the gateway and keeps identity logic out of the reports service.

For client requests, the gateway wraps service calls with timeout and retry behavior. Report creation and report updates use longer timeouts because they may include Cloudinary file upload. General report queries and user queries use shorter timeouts. When a service error occurs, the gateway returns a controlled error object with the service name and error details.

Table 4.2: Comparison of message brokers for the VietFlood backend.

Criteria	**RabbitMQ**	**Redis Pub/Sub**	**Kafka**
Request-response work	Fits gateway-to-service commands with queues and acknowledgements.	Fast, but less suitable for durable request handling.	Heavier than needed for this prototype.
Operational cost	Runs locally through Docker Compose and is simple enough for two services.	Also simple locally, but less aligned with NestJS queue patterns.	Requires more setup and topic management.
VietFlood usage	Used for auth and report service messages.	Used separately through Redis for caching.	Not used in this project.

Figure 4.1: RabbitMQ communication flow between the API gateway, auth service, reports service, auth queue, and reports queue.

4.3 Authentication and Authorization Implementation

VietFlood uses username-password authentication with access tokens and refresh tokens. Passwords are hashed with bcrypt before storage [26]. After sign-in, the backend issues an access token for authenticated requests and a refresh token for session renewal. The access token follows the JSON Web Token format [27]. The access token payload contains the user ID, username, role, first name, and last name. The role is used by the gateway to protect citizen, relief, and admin routes.

4.3.1 Authentication Mechanism Implementation

a) Registration Process

Registration is handled by the auth service. The service first checks whether the email, username, or phone number already exists. If one of these values is already used, the service rejects the request. If the values are available, the password is hashed with bcrypt using a salt round value of 10. The resulting user record is saved in the `users` table.

The mobile app sends registration data through the API gateway. The user provides email, username, password, phone number, name fields, date of birth, and address fields. These fields match the user entity in the backend. The registration result is intentionally small: the backend returns a success message instead of exposing the stored user record.

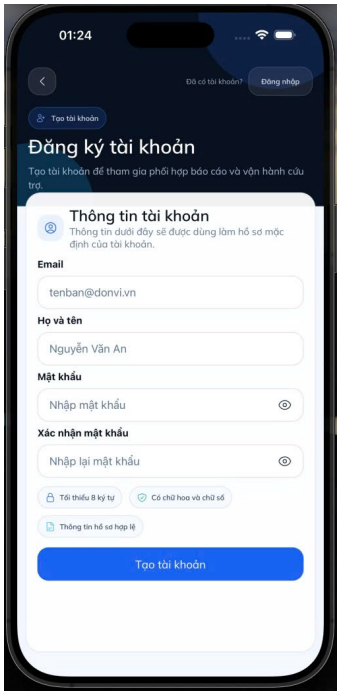

Figure 4.2: Citizen registration screen in the VietFlood mobile app.

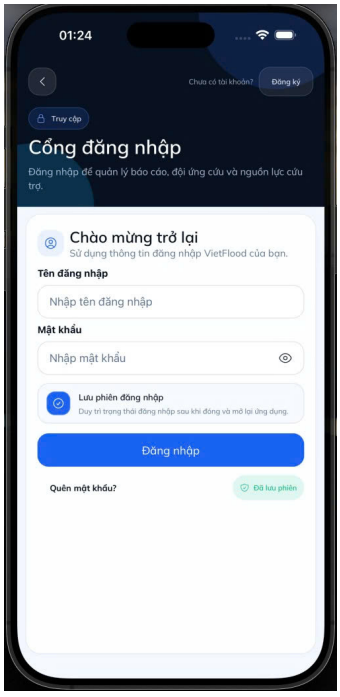

Figure 4.3: Citizen login screen in the VietFlood mobile app.

b) Login Process

Login is handled through the `/auth/sign_in` endpoint. The auth service looks up the user by username and compares the submitted password with the stored bcrypt hash. If the password is valid, the service signs an access token and asks the refresh-token service to create a refresh token.

The mobile app and web dashboard handle the login response differently. The mobile app can store tokens in Expo SecureStore or keep them only for the current app session. The web dashboard stores access and refresh tokens in browser storage and loads the admin profile before allowing access to the dashboard. If the profile is not an admin profile, the dashboard blocks the admin workspace.

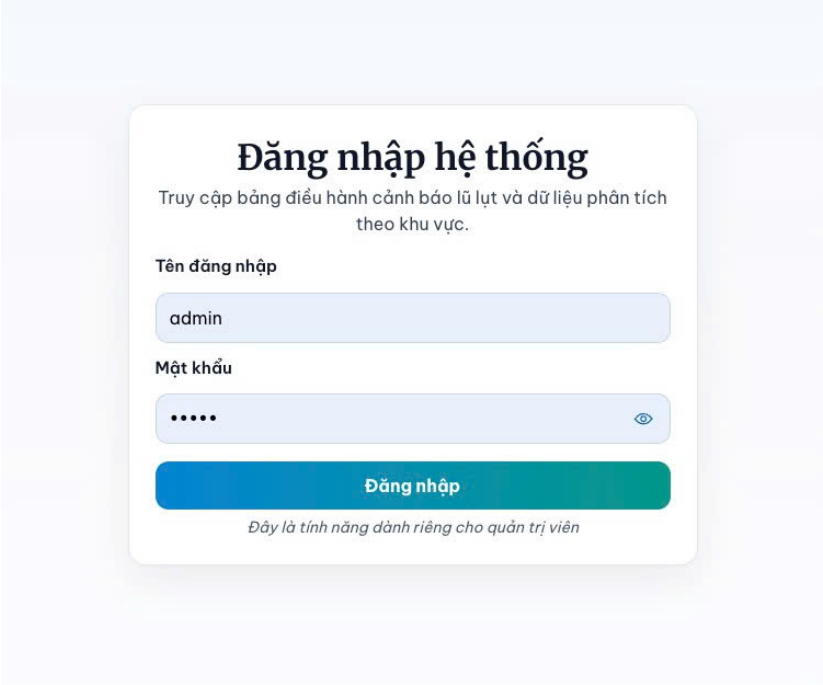

Figure 4.4: Admin login page in the VietFlood web dashboard.

4.3.2 Refresh Token Implementation

The refresh-token service stores hashed refresh tokens in the `refresh_tokens` table. Each refresh token belongs to one user. When a user signs in again, the existing refresh token row can be updated instead of creating unlimited token rows. The service also stores expiration time and revoked time. This lets the backend reject expired or revoked tokens.

The mobile app refreshes access tokens through `/auth/refresh_token`. The token storage helper preserves the user's storage choice. If the user used a session-only login, a refreshed token stays session-only. If the user saved the login, the refreshed token is written back to SecureStore.

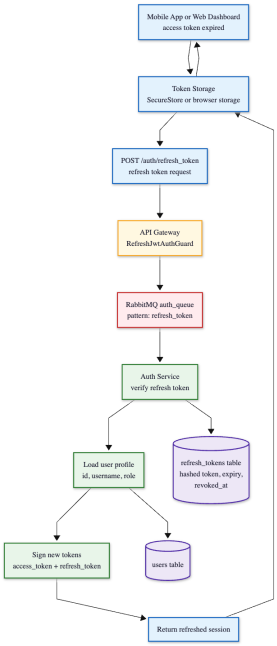

Figure 4.5: Refresh-token flow from mobile or web client through the API gateway, auth service, refresh-token table, and new access token response.

4.3.3 Authorization Process Implementation

Authorization is enforced at the API gateway. JWT guards validate the token. Role guards check whether the route accepts citizen, relief, or admin users. Citizen users can create and manage their own reports. Relief users can read reports and inspect locations for response work. Admin users can review evidence, update status, and manage users through the dashboard.

This split is important because relief users should not verify or reject reports in the current permission model. Verification, rejection, and resolution are admin-side actions. Relief screens are built around field awareness: report list, report details, map view, and citizen location support.

4.4 Report Processing and Evidence Upload Implementation

Report creation is the central workflow in VietFlood. The mobile app builds a multipart request with report fields and optional files. The API gateway receives the request through the reports controller. File buffers are uploaded to Cloudinary in the `vietflood/reports` folder [8]. The gateway then sends the report data and evidence metadata to the reports service through RabbitMQ.

The reports service stores the record in PostgreSQL. Category values are normalized as arrays. Evidence entries are saved as JSONB values with URL, public ID, and resource type [15]. Image URLs are also copied into the report image list for easier display by clients. After saving the report, the service writes a Redis cache entry using the report ID [16].

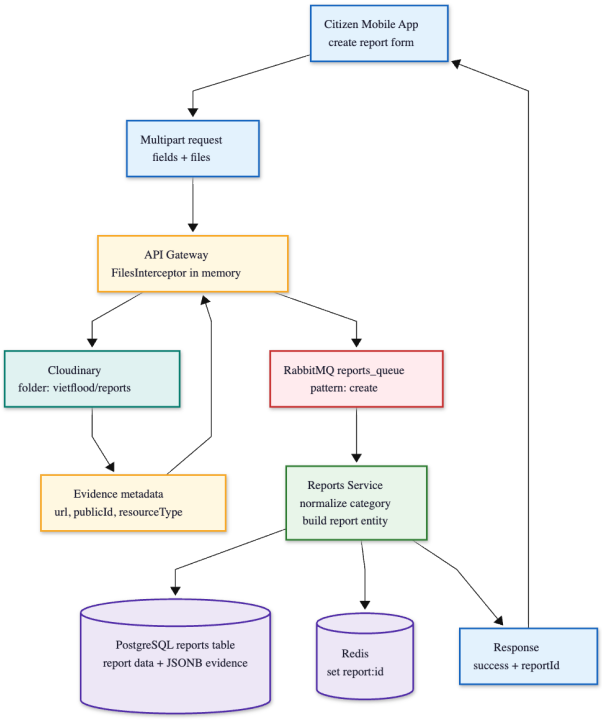

Figure 4.6: Report creation flow from mobile form submission to Cloudinary upload, RabbitMQ message, PostgreSQL insert, and Redis cache write.

4.4.1 Report Update and Delete Implementation

Report update uses the same evidence handling path as report creation. If new evidence files are attached, the gateway uploads them to Cloudinary and appends the returned metadata to the update payload. The reports service checks that the report belongs to the given user ID, merges new evidence with existing evidence, updates image URLs, deletes the old cache entry, and writes a refreshed cache entry.

Report deletion also checks ownership. Before deleting the report row, the service reads Cloudinary public IDs from the stored evidence metadata. If public IDs exist, it calls the Cloudinary bulk delete helper. After the media deletion request, it removes the report row from PostgreSQL and deletes the Redis cache entry.

4.5 Public Library Implementation

The shared package is implemented as `vietflood_common`. It contains infrastructure modules that are reused by the gateway and services. This package keeps the backend code smaller because Redis, Cloudinary, and logging behavior are not rewritten in each service.

The Cloudinary service supports buffer upload, file path upload, base64 upload, single-file delete, bulk delete, rename, optimized image URL creation, and raw URL creation. The report gateway uses buffer upload for multipart report evidence. The reports service uses bulk delete when a report is removed.

The Redis service wraps common operations: `set`, `get`, `del`, and `publish` [16]. In VietFlood, Redis is used mainly for report caching. The logger service emits structured logs with service name and request context so backend behavior can be traced during development and deployment checks.

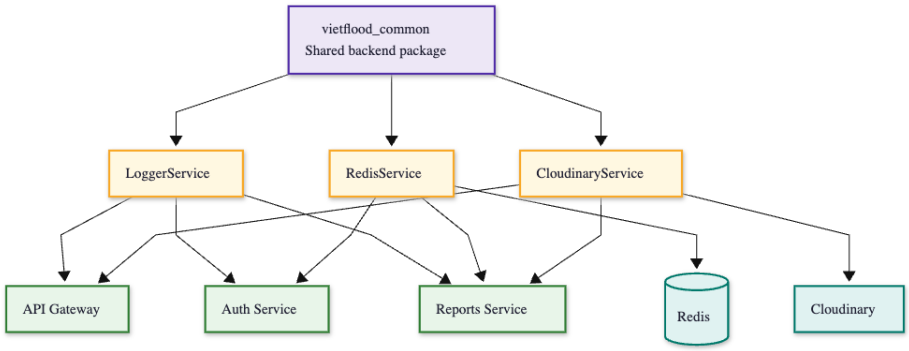

Figure 4.7: Shared common library structure showing LoggerService, RedisService, CloudinaryService, and the backend modules that import them.

4.6 Real-Time Tracking Implementation

VietFlood includes a Socket.IO gateway for live location updates [17]. The client event is `send-location`. Its payload contains latitude, longitude, accuracy, heading, speed, and timestamp. The gateway validates latitude and longitude ranges before broadcasting the data. If the location is invalid, the socket receives `location-error`. If the payload is valid, all connected clients receive `receive-location`.

The gateway also handles connection and disconnection events. When a socket disconnects, the gateway emits `user-disconnected`. The feature is useful for rescue-oriented flows because a relief screen can receive a citizen's latest location while the report is being handled. The current implementation proves the live event path; later work can add authenticated sockets and persistent location history.

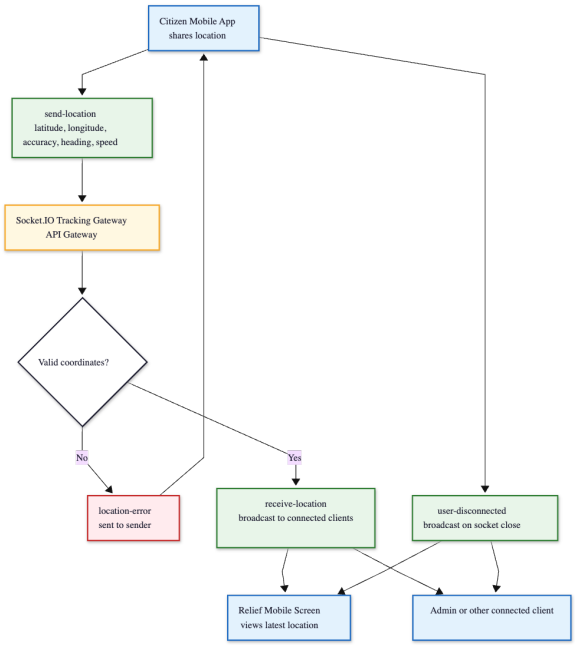

Figure 4.8: Socket.IO live tracking flow with send-location, receive-location, location-error, and user-disconnected events.

4.7 Mobile Citizen Feature Implementation

4.7.1 Report List and Profile Access

The mobile application provides the main citizen workflow. After login, the app stores tokens and loads the current profile. The profile screen calls the authenticated profile endpoint, while report screens call the report service wrapper. The report service normalizes backend records before rendering them because the backend and clients use a mix of snake case and camel case fields.

The citizen report list is loaded from the backend and displayed with status, category, location, and evidence information. Status values are normalized so older or alternate values can still be shown in the app. This reduces UI failure when an API response contains a value such as `resolved` while the local screen expects a status label.

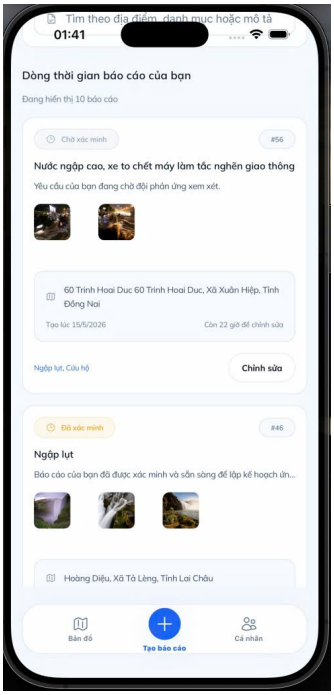

Figure 4.9: Citizen report list or citizen home screen showing submitted reports, status labels, and report location information.

4.7.2 Create Report Implementation

The create-report screen collects category, description, address fields, coordinates, urgency, severity, and evidence images. The mobile app uses province and ward data from the Vietnam Provinces Open API [7]. If device location permission is available, the app can prefill latitude and longitude. The form is sent as multipart data so the backend can receive both text fields and files.

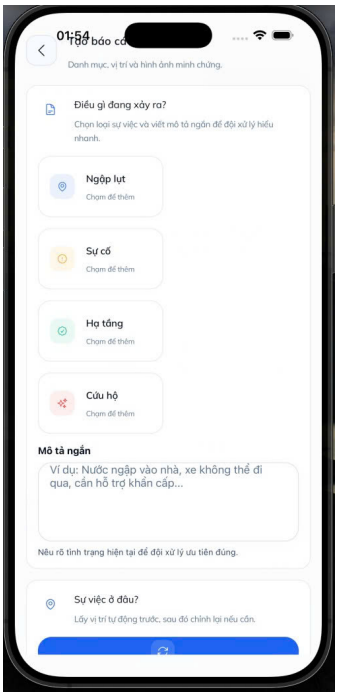

Figure 4.10: Create report form in the VietFlood mobile app.

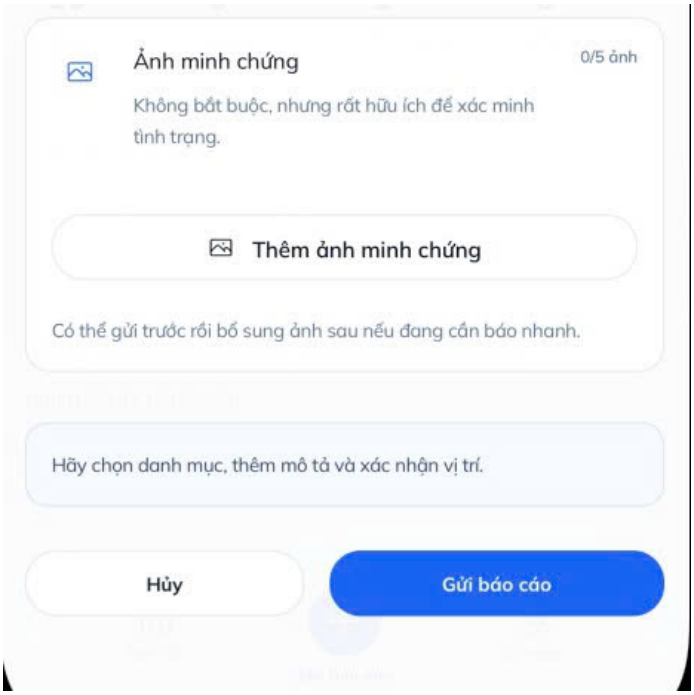

Figure 4.11: Evidence upload in the VietFlood mobile app.

4.8 Relief Feature Implementation

The relief screens focus on reading and locating reports. The relief report list shows submitted cases. The detail screen shows description, category, evidence, status, and address fields. The map screen helps response staff understand where reports are located. The relief role is intentionally narrower than the admin role: it supports response coordination, not final report verification.

The mobile app includes a dedicated relief directions map component. This component is used to display direction-oriented information for response work. It relies on the normalized report location fields produced by the report service wrapper, so a report can be displayed even when coordinates arrive as either `lat`/`lng` or `latitude`/`longitude`.

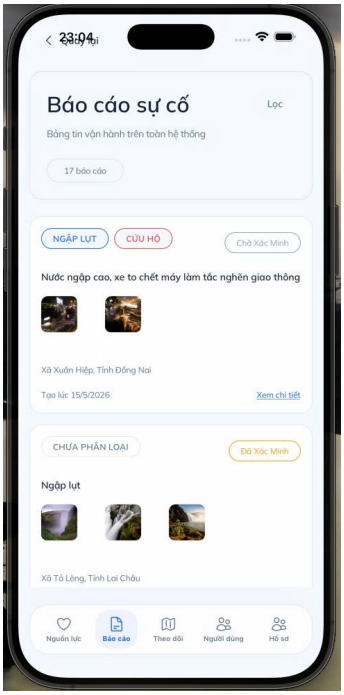

Figure 4.12: Relief report list in the mobile app.

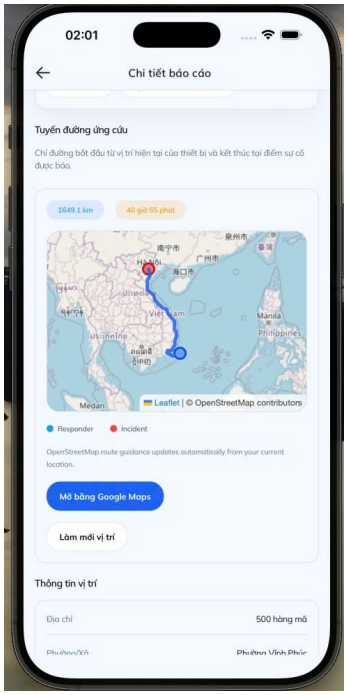

Figure 4.13: Relief map screen in the mobile app.

4.9 Web Dashboard Implementation

The web dashboard is used by administrators. It begins at the admin login page and checks the current profile before displaying protected dashboard content. The reports overview component loads reports from the backend, formats dates for Vietnamese display, translates category and status values, extracts thumbnail images from evidence, and shows reporter information when it is available.

The dashboard supports search and status filters. Search uses report content such as reporter name, account name, phone number, location, and description. Status filtering supports `pending`, `verified`, `resolved`, and `rejected`. The dashboard also provides status update controls. These controls call the backend route for admin report updates.

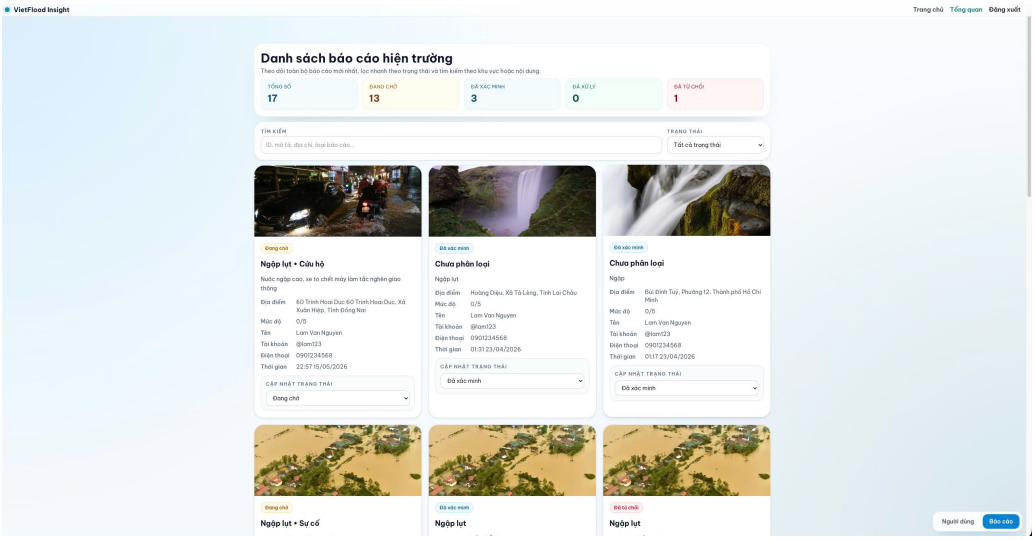

Figure 4.14: Admin dashboard overview with report statistics, search, status filters, report cards, and evidence thumbnails.

4.9.1 Evidence Review and Status Update

Evidence review is based on the stored Cloudinary URLs. The dashboard chooses a thumbnail from the evidence list first, then falls back to the image list. This makes the UI tolerant of reports created by different versions of the client. When an administrator changes a report status, the dashboard sends the update to the gateway. The backend then forwards the update to the reports service through RabbitMQ.

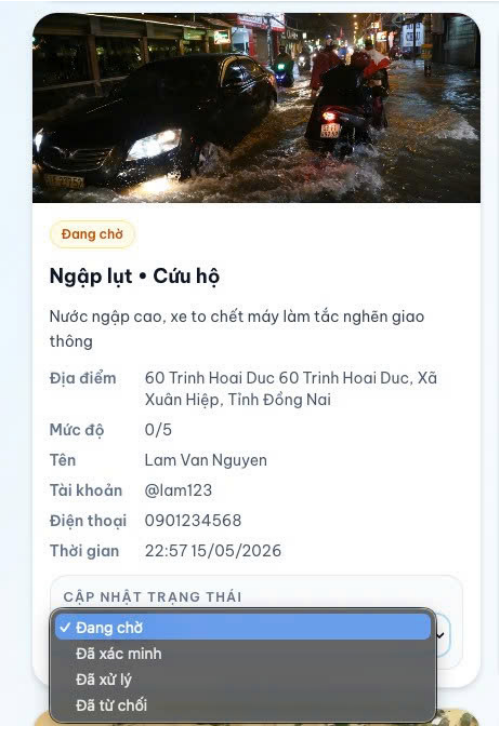

Figure 4.15: Admin evidence review and status update screen for pending, verified, resolved, and rejected reports.

4.10 Deployment Implementation

The backend deployment is based on Docker images and a multi-container Azure App Service setup [19]. Local Docker Compose builds the API gateway, auth service, and reports service from source [18]. It also starts RabbitMQ and Redis. Production Compose uses Docker Hub images for the three backend services. The same service names and queue names are used so local and deployed behavior stay close.

The GitHub Actions workflow builds images for the gateway, auth service, and reports service [20]. After pushing the images to Docker Hub, it configures the Azure App Service with the Compose file and restarts the application. This deployment flow belongs in the implementation chapter because it describes how the built services are packaged and run. The same idea of repeatable build and release steps is a central theme in continuous delivery and DevOps practice [28, 29].

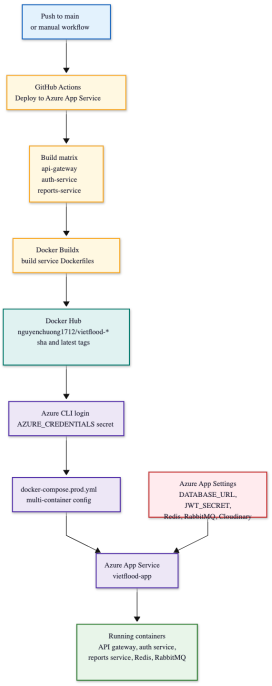

Figure 4.16: CI/CD workflow from GitHub push to Docker image build, Docker Hub push, Azure App Service configuration, and application restart.

4.11 User Management Implementation

User management is handled by the auth service and exposed through the API gateway. The auth service can list users, load a user by ID, update a user profile, update a user by ID, and delete a user. The user entity stores role information, so the same user table supports citizens, relief users, and administrators.

The web dashboard uses profile validation to protect admin-only screens. User-oriented mobile screens use authenticated profile calls and update calls to let users manage their own information. The current project keeps role management simple. Roles are stored as enum values in the database, and route access is enforced by backend guards.

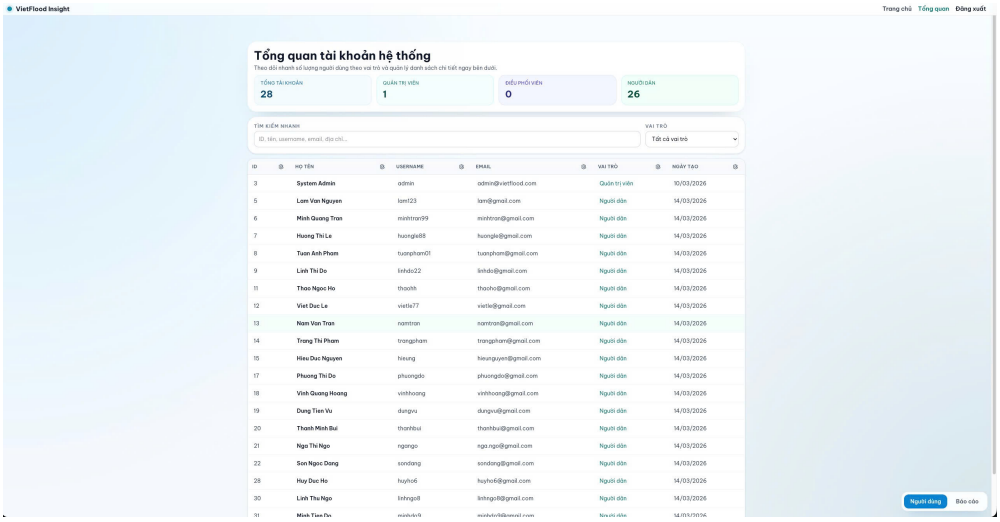

Figure 4.17: User management view showing citizen, relief, and admin accounts with profile and role information.

Chapter 5
Evaluation

This chapter evaluates the VietFlood prototype against the project goals stated in the proposal and the implementation described in the previous chapter. The evaluation is based on the current source code in the backend, mobile application, web dashboard, and shared library. It focuses on behavior that can be traced in the implementation: report creation, evidence upload, admin review, relief access, authentication, service communication, data storage, and deployment readiness.

The project has not yet gone through a field trial with citizens or relief teams. Because of that, this chapter does not claim measured response-time gains, public adoption, or operational impact during a real flood event. The evaluation is limited to implementation review and functional scenario checking. That boundary matters. VietFlood is a working prototype, but a disaster-response system needs stronger testing before it can be used in production.

5.1 Evaluation Scope

The evaluation covers three user roles. A citizen can register, sign in, create a flood report, attach evidence, and review submitted reports. A relief user can view reports and use location-oriented screens for response work. An admin can access the web dashboard, inspect reports, review evidence, update report status, and manage users.

The backend is evaluated through the responsibilities of each service. The API gateway is the public REST and Socket.IO entry point. The authentication service handles users, login, profile data, refresh tokens, logout, and user management. The reports service handles report records, evidence metadata, status values, caching, and deletion. RabbitMQ connects the gateway with the two backend services. PostgreSQL stores durable data. Redis stores report cache entries. Cloudinary stores uploaded evidence files.

The mobile and web clients are evaluated as separate interfaces over the same backend. The mobile application uses Expo React Native and supports citizen and relief workflows [5, 6]. The web dashboard uses Next.js and is focused on admin review [21]. This split matches the way VietFlood is expected to be used: citizens and relief users work from phones, while admins review incoming reports from a dashboard.

5.2 Functional Evaluation

The first evaluation question is simple: can a report move from a citizen phone to the backend and then to the admin dashboard with enough information for review? In the current implementation, that path is present. The citizen report form sends category, description, address, coordinates, urgency, severity, and optional evidence files. The API gateway uploads files to Cloudinary before sending the cleaned report payload to the reports service through RabbitMQ. The reports service then stores the report in PostgreSQL and writes a Redis cache entry.

The admin dashboard reads the report list from the same backend. It can show status values, reporter details, report location fields, evidence thumbnails, urgency, and sever-

ity. It can also update the status to values such as `pending`, `verified`, `resolved`, and `rejected`. This is the main review loop of VietFlood.

Table 5.1: Functional evaluation of VietFlood user flows.

Scenario	**Expected behavior**	**Evaluation result**
Citizen account access	A citizen can register, sign in, and load a profile after authentication.	The auth service supports registration, sign-in, profile loading, and profile updates. Passwords are stored with bcrypt hashing, and authenticated requests use JWT access tokens.
Citizen report creation	A citizen can submit a flood-related report with location and optional evidence.	The mobile app builds the report request, the gateway accepts multipart evidence files, Cloudinary stores the media, and the reports service stores report data with evidence metadata.
Citizen report history	A citizen can see reports linked to the signed-in account.	The backend has a user-report retrieval path, and the mobile app normalizes backend fields such as latitude, longitude, address, category, and status for display.
Admin report review	An admin can inspect reports and update status after checking the evidence.	The web dashboard loads reports, filters by status, shows evidence thumbnails, and calls the admin update endpoint for status changes.
Relief report access	A relief user can view report lists, inspect report details, and work with report locations.	The relief mobile screens focus on report awareness, map viewing, and route-ready reports. The current design does not give relief users permission to verify or reject reports.
Token refresh and logout	A signed-in user can keep a session active without entering credentials again, and can end the session.	The backend provides refresh-token and logout flows. The mobile app preserves the user's storage choice when refreshing tokens, while the web dashboard refreshes tokens from browser storage.
Live location events	A client can send live coordinates, and other clients can receive updates.	The Socket.IO gateway validates latitude and longitude ranges before broadcasting location events. Invalid coordinates return a location error instead of being broadcast.

One point needs to be separated from the table. The mobile relief interface contains some status-oriented UI concepts, but the current permission model keeps report verification under the admin role. This is a good boundary for the prototype. Relief teams need fast access to report locations and citizen details, but report approval affects the trust level of the whole system. Keeping verification in the admin dashboard reduces the chance that field users accidentally change the public state of a report while they are working under pressure.

5.3 Backend Evaluation

The backend structure fits the project size. Splitting the system into an API gateway, auth service, and reports service keeps the two largest business areas apart: identity and report management. This evaluation follows the same architectural concern described by software architecture literature: separate responsibilities should serve system qualities such as maintainability, deployability, and modifiability [24]. The gateway handles client-facing details such as HTTP routes, file upload parsing, JWT guards, role checks, and Socket.IO events. The services keep the data work behind RabbitMQ message patterns.

RabbitMQ is useful in this project because report creation has more than one step [13]. A report can include images or videos. The gateway first uploads the files to Cloudinary, then sends the report payload to the reports service. The service writes the final record to PostgreSQL and updates Redis. If the reports service cannot respond in time, the gateway returns a controlled error instead of leaving the client waiting without feedback. The timeout and retry behavior in the gateway is a practical choice for a prototype with more than one service.

PostgreSQL is a good fit for the current data model. Users, refresh tokens, and reports all have structured fields and clear relations. Reports also need flexible evidence metadata, and the implementation stores that metadata as JSONB [15]. This avoids creating a separate evidence table at the prototype stage while still keeping the public URL, Cloudinary public ID, and resource type with the report.

Redis is used in a narrow way [16]. The reports service writes cache entries such as `report:id` after report creation and update, and deletes those entries when reports change. This is enough for the current system. It keeps the cache logic easy to reason about. Later, if the report list becomes large or if map views query reports by province or status, the cache strategy may need to include list-level or geospatial keys.

The shared library, `vietflood_common`, reduces repeated infrastructure code. The Cloudinary module supports buffer upload, file upload, base64 upload, delete, bulk delete, rename, and URL helpers. The Redis module wraps common cache operations. The logger keeps service logs in one style. For a microservice project, this is a useful trade: shared code is kept to infrastructure concerns, while each service still owns its business logic.

5.4 Client Evaluation

The mobile application is the most important client for VietFlood because flood reporting happens in the field. The implementation supports account access, profile screens, report creation, report lists, evidence upload, location prefill, and relief-oriented report views. It also uses the Vietnam Provinces Open API for province and ward selection, which keeps address input closer to local administrative divisions.

Token handling on mobile is handled with care. The app can keep tokens in Expo SecureStore when the user wants persistence, or keep tokens only for the current session. When the app refreshes a token, it keeps the same storage mode. This is a small implementation detail, but it matters on shared phones or temporary devices.

The web dashboard is narrower than the mobile app. That is acceptable because the dashboard has a different job. It is built for admin review rather than field reporting. The report overview screen loads backend reports, formats statuses, shows evidence thumbnails, filters reports, displays reporter information, and sends admin status updates. The dashboard also validates the profile after login, so non-admin users are blocked from the

admin workspace.

There are still inconsistencies between clients and backend naming. The mobile app normalizes values such as `lat/lng` and `latitude/longitude`, and also handles snake case and camel case fields. This normalization protects the interface from small backend changes, but it also shows that the API contract should be cleaned before a larger release. A stable response shape would reduce client-side mapping code.

5.5 Security and Access Control Evaluation

VietFlood uses role-based access control through the gateway. Citizen, relief, and admin routes are separated by JWT guards and role guards. This gives the project a clear permission model. Citizens can create and manage their own reports. Relief users can inspect reports for response work. Admin users can update status values and manage users.

Password handling is implemented with bcrypt [26]. Access tokens carry the user ID, username, role, first name, and last name through JWT claims [27]. Refresh tokens use a separate secret and are stored in the database with expiration and revocation data. This gives the backend a way to end sessions without waiting for all tokens to expire naturally.

There are areas that should be reviewed before production use. Refresh-token storage and comparison should be tested carefully to confirm that hashed tokens are validated in the intended way. Token rotation should also be tightened so repeated refresh requests cannot keep an old token alive longer than expected. The Socket.IO tracking gateway currently validates coordinate ranges, but live tracking should also be tied more strictly to authenticated users and role permissions before it is used in the field.

Evidence upload also needs guardrails. Cloudinary handles file storage, but the application should still enforce file count, file size, file type, and scanning rules at the gateway level. The current implementation is enough for prototype review, but public evidence upload can become a spam and storage-cost problem if it is opened without limits.

5.6 Deployment and Operations Evaluation

The deployment setup is clear enough for a student project and practical enough for a prototype demonstration. Local development can run the backend services with Docker Compose together with RabbitMQ and Redis [18]. The production workflow builds Docker images for the API gateway, auth service, and reports service, pushes those images to Docker Hub, and deploys the multi-container stack to Azure App Service [20, 19].

The deployment design keeps runtime values in environment variables. Database URL, Redis settings, RabbitMQ credentials, JWT secrets, refresh-token secret, Cloudinary credentials, and admin seed values are not hardcoded in the application. This is the correct direction for a system that may run in different environments.

The current operational weakness is observability. The shared logger gives the services a consistent logging style, but the system still needs health checks, error dashboards, alerting, and deployment rollback notes. A flood reporting system must be easy to diagnose when reports stop appearing or when file uploads fail. Logs are a start, but operations work should continue beyond logs.

5.7 Evaluation Result

The evaluation shows that VietFlood meets the main prototype goals. A citizen can submit a report with location and evidence. The backend stores the report and evidence metadata. The admin dashboard can inspect reports and change their status. Relief users can read report data and work from mobile screens that focus on location awareness. The backend is also divided into services in a way that matches the problem: authentication, reports, shared infrastructure, and real-time tracking.

The system is not finished as an operational disaster-response platform. It still needs load testing and user testing. Upload controls, tracking authentication, API contracts, and monitoring also need more work. Field validation is the largest missing evidence. The code proves that the main software path works; it does not yet prove that the system will hold up during a real flood, with poor network conditions and many reports arriving at the same time.

For the thesis prototype, this result is acceptable. VietFlood demonstrates that a multi-platform flood reporting system can be built with a NestJS microservice backend, a React Native mobile client, a Next.js admin dashboard, PostgreSQL, Redis, RabbitMQ, Cloudinary, Docker, and Azure deployment. The next work should test the current workflows with real users, real devices, slower networks, and larger report volume.

Chapter 6
Conclusion and Future Works

6.1 Conclusion

This thesis presented VietFlood, a multi-platform flood reporting prototype designed for citizen reporting, relief awareness, and admin review. The project began from a practical problem: flood information often reaches communities through scattered messages, photos, and location shares. That information can be useful, but it is hard to organize when many reports arrive at the same time. VietFlood gives that process a clearer path. A citizen can submit a report from the mobile app, attach evidence, provide location details, and let the backend store the report for later review.

The completed prototype connects four main parts. The NestJS backend provides the API gateway, authentication service, reports service, and live tracking gateway. The Expo React Native mobile app supports citizen and relief workflows. The Next.js web dashboard supports admin login, report filtering, evidence preview, and status updates. The shared `vietflood_common` package keeps Redis, Cloudinary, and logging utilities in one place so the backend services do not repeat the same infrastructure code.

The implementation shows that the main report loop works as a software system. The API gateway accepts report submissions and evidence files. Cloudinary stores uploaded media. PostgreSQL stores users, refresh tokens, and report data. Redis stores report cache entries. RabbitMQ moves backend requests from the gateway to the auth and reports services. Socket.IO supports live location updates with coordinate validation. This design follows the microservices pattern of separating independent capabilities into owned services while using message brokers for communication [33, 34]. These parts do not solve every disaster-response problem, but they prove that the prototype can move a citizen report from mobile submission to backend storage and admin review.

Role separation is one of the clearer outcomes of the system. Citizens create and manage their own reports. Relief users read report data and use mobile screens focused on response awareness. Admin users review reports and update report status. This separation keeps verification and rejection under admin control, while relief users can still inspect reports and locations for field work. That boundary is useful because report status affects trust in the system.

The project also shows the value of building the system across mobile, web, backend, and shared code rather than treating flood reporting as only a mobile form. The mobile app is close to the citizen and relief user. The web dashboard gives admins a larger view for review. The backend holds the rules, storage, messaging, and role checks. The shared library keeps repeated infrastructure code under control. Together, these parts make VietFlood a working foundation for later testing and improvement.

At the same time, VietFlood should be understood as a prototype. It has not yet been tested in a real flood situation, with many users, unstable network connections, and urgent field decisions. The current system demonstrates the technical path, not full operational readiness. Before practical use, it needs field testing, stronger upload rules, better monitoring, stricter live-tracking authentication, and more careful review of token behavior.

6.2 Future Works

Future work should begin with field testing. The system needs feedback from citizens, relief workers, and administrators using real phones and realistic network conditions. That testing would show whether the report form is clear, whether evidence upload is reliable enough, and whether the admin dashboard provides enough context for status review.

The mobile app should also support poor-network conditions better. A future version should let citizens save reports locally, retry submission when the connection returns, and compress evidence files before upload. The app should also make the report state clear: sent, saved, uploading, or failed.

Report review can be improved through duplicate detection, spam handling, and evidence quality checks. Reports with similar coordinates, descriptions, and creation times could be grouped for admin review. Relief workflows could also add assignment, response notes, and route history while keeping verification and rejection under admin control.

Security and operations should be strengthened before public deployment. Refresh-token handling, file size limits, file type checks, live-tracking authentication, audit logs, health checks, service alerts, and deployment rollback steps all need more work. The operational improvement should follow the DevOps and deployment automation practices documented in modern infrastructure literature [29, 35]. If report storage or evidence upload fails, administrators should know quickly through proper monitoring and alerting. The containerized deployment using Docker already provides a foundation, but observability, alerting, and automated recovery are essential before production use [18].

In later research, VietFlood could also support flood insight analysis. The existing reports already contain category, urgency, severity, coordinates, address fields, status, and timestamps. With enough verified reports, the system could generate simple heat maps, time-based charts, or province-level summaries. These features should come after the reporting path is stable. Bad input will produce bad insight, so data quality must come first.

Bibliography

[1] M. F. Goodchild and J. A. Glennon, "Crowdsourcing geographic information for disaster response: a research frontier," *International Journal of Digital Earth*, vol. 3, no. 3, pp. 231–241, 2010.

[2] M. Imran, C. Castillo, F. Diaz, and S. Vieweg, "Processing social media messages in mass emergency: A survey," *ACM Computing Surveys*, vol. 47, no. 4, pp. 67:1–67:38, 2015.

[3] NestJS, "Nestjs documentation." `https://docs.nestjs.com/`, 2026.

[4] TypeScript, "Typescript documentation." `https://www.typescriptlang.org/docs/`, 2026.

[5] Expo, "Expo documentation." `https://docs.expo.dev/`, 2026.

[6] Meta Open Source, "React native documentation: Introduction." `https://reactnative.dev/docs/getting-started`, 2026.

[7] Vietnam Open API, "Province open api." `https://provinces.open-api.vn/`, 2026.

[8] Cloudinary, "Upload api reference." `https://cloudinary.com/documentation/image_upload_api_reference`, 2026.

[9] M. Fowler and J. Lewis, "Microservices: a definition of this new architectural term." `https://martinfowler.com/articles/microservices.html`, 2014.

[10] J. Thönes, "Microservices," *IEEE Software*, vol. 32, no. 1, p. 116, 2015.

[11] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, pp. 195–216, Springer International Publishing, 2017.

[12] NestJS, "Microservices documentation." `https://docs.nestjs.com/microservices/basics`, 2026.

[13] RabbitMQ, "Consumer acknowledgements and publisher confirms." `https://www.rabbitmq.com/docs/3.13/confirms`, 2026.

[14] TypeORM, "Typeorm documentation: Getting started." `https://typeorm.io/docs/getting-started/`, 2026.

[15] PostgreSQL Global Development Group, "Postgresql documentation: Json types." `https://www.postgresql.org/docs/current/datatype-json.html`, 2026.

[16] Redis, "Redis documentation." `https://redis.io/docs/latest/`, 2026.

[17] Socket.IO, "Socket.io documentation." `https://socket.io/docs/v4/`, 2026.

[18] Docker, "Docker compose documentation." `https://docs.docker.com/compose/`, 2026.

[19] Microsoft, "Continuous deployment with custom containers in azure app service." https://learn.microsoft.com/en-us/azure/app-service/deploy-ci-cd-custom-container, 2026.

[20] GitHub, "Github actions documentation." https://docs.github.com/en/actions, 2026.

[21] Vercel, "Next.js documentation." https://nextjs.org/docs, 2026.

[22] React, "React documentation." https://react.dev/, 2026.

[23] Tailwind Labs, "Tailwind css documentation." https://tailwindcss.com/docs, 2026.

[24] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley Professional, 4th ed., 2021.

[25] A. Videla and J. Williams, *RabbitMQ in Action: Distributed Messaging for Everyone*. Manning Publications, 2012.

[26] N. Provos and D. Mazières, "A future-adaptable password scheme," in *1999 USENIX Annual Technical Conference*, USENIX Association, 1999.

[27] M. B. Jones, J. Bradley, and N. Sakimura, "Json web token (jwt)," RFC 7519, Internet Engineering Task Force, 2015.

[28] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.

[29] G. Kim, P. Debois, J. Willis, and J. Humble, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, 2016.

[30] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.

[31] PostgreSQL Global Development Group, "Postgresql documentation." https://www.postgresql.org/docs/, 2026.

[32] Redux Toolkit, "Redux toolkit documentation." https://redux-toolkit.js.org/, 2026.

[33] C. Richardson, *Microservices patterns: With examples in Java*. Manning Publications, 2018.

[34] S. Newman, *Building microservices: Designing fine-grained systems*. O'Reilly Media, 2nd ed., 2021.

[35] J. Turnbull, *The Docker Book: Containerization is the new virtualization*. Turnbull Press, 2014.

Appendix A
API Specifications

This appendix provides representative API response structures for the VietFlood system. Use recorded Postman responses or logged API traffic to replace placeholders where available.

A.1 Authentication API

Endpoint: `POST /auth/sign_in`

Response:

```
{
  "statusCode": 200,
  "message": "Login successful",
  "data": {
    "accessToken": "<JWT_ACCESS_TOKEN>",
    "refreshToken": "<JWT_REFRESH_TOKEN>",
    "user": {
      "id": "<uuid>",
      "email": "<user_email>",
      "role": "citizen"
    }
  }
}
```

Listing A.1: Sign In Response

A.2 Report Creation API

Endpoint: `POST /reports`

Response:

```
{
  "statusCode": 201,
  "message": "Report created successfully",
  "data": {
    "id": "<report_id>",
    "category": ["flood"],
    "description": "<description>",
    "status": "pending",
    "evidence": [
      {
        "url": "<cloudinary_url>",
        "resourceType": "image"
      }
    ]
  }
}
```

Listing A.2: Create Report Response

Appendix B
Interface Screenshots

Key user interface screens are shown here. These images support the main report without repeating the implementation details.

B.1 Mobile Screens

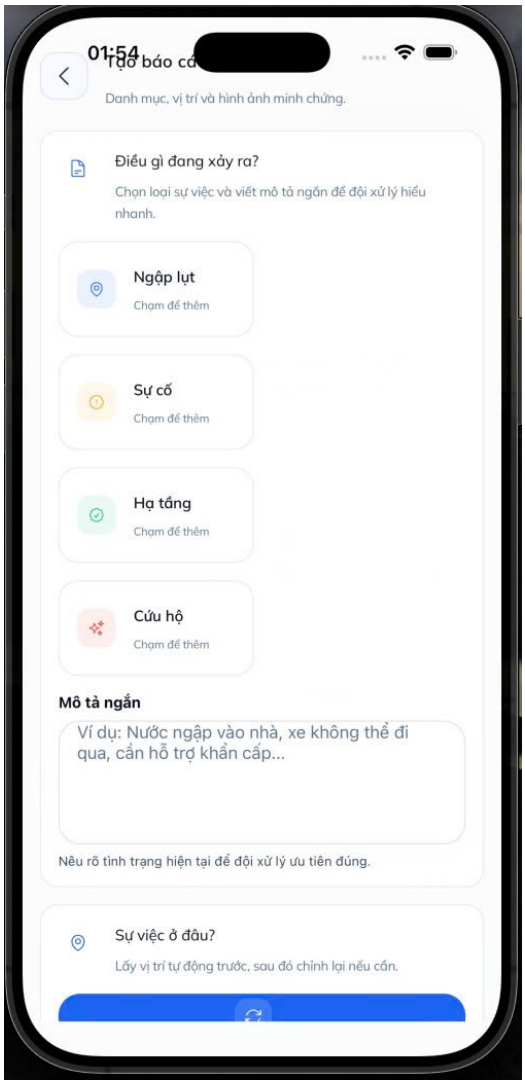

Figure B.1: Citizen report creation screen.

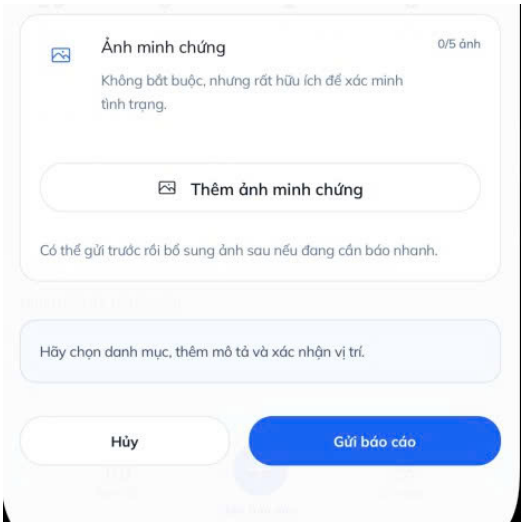

Figure B.2: Evidence upload screen on the mobile app.

B.1.1 Relief and Admin Screens

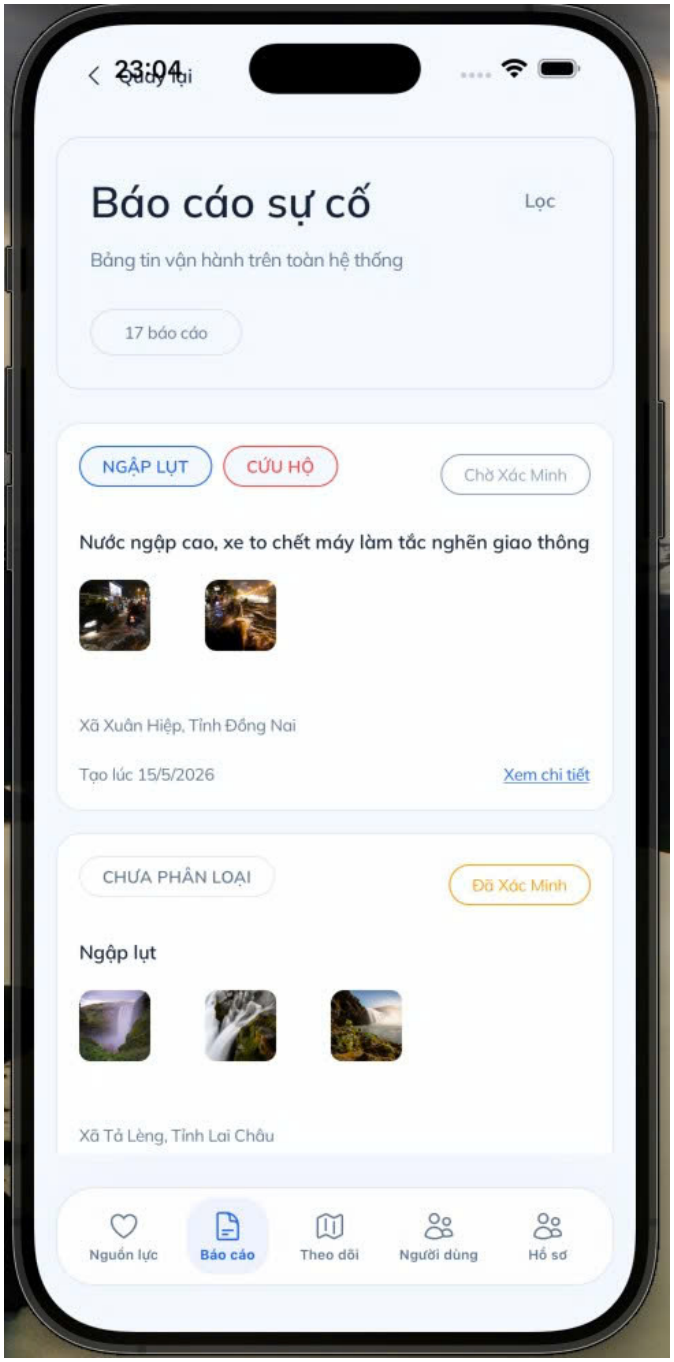

Figure B.3: Relief worker report list screen.

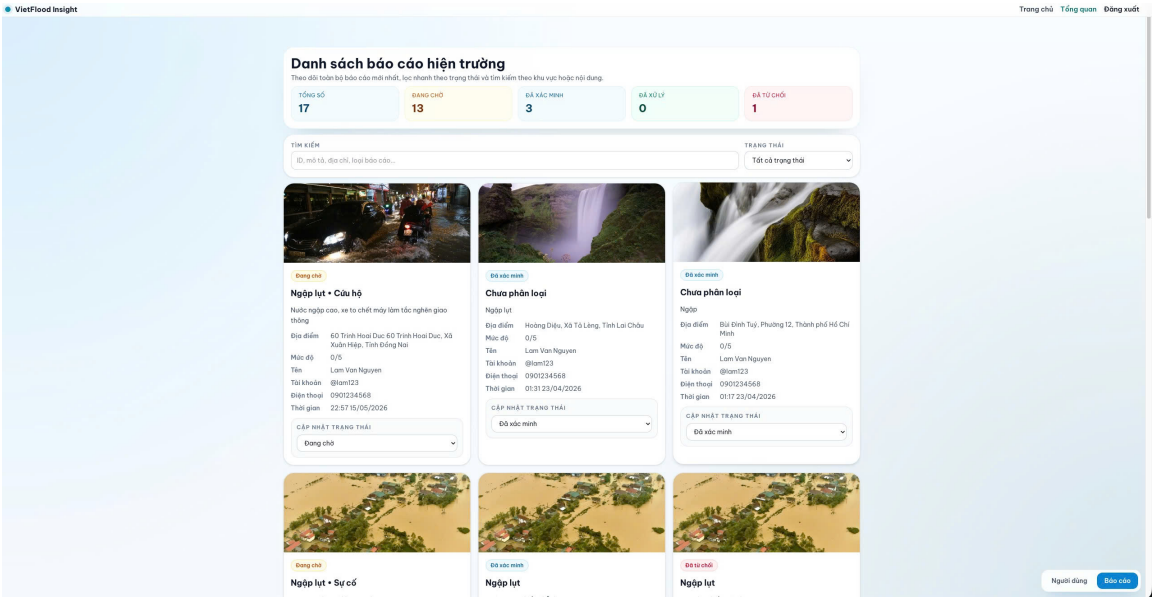

Figure B.4: Admin dashboard overview.

Appendix C
Test Cases

This table summarizes the most important validation flows for the VietFlood prototype.

Table C.1: Key VietFlood Test Cases

Scenario	**Expected Result**	**Status**
User login	Access token and user profile are returned	To be completed
Create report with evidence	Report saved and evidence links stored	To be completed
View user reports	Citizen can view own report list	To be completed
Admin verify/reject report	Report status updates correctly	To be completed
Relief user views map	Map shows report locations and markers	To be completed
Refresh token flow	New access token issued without login	To be completed

Appendix D
Deployment and Configuration

This chapter summarizes the deployment configuration groups used by VietFlood. Sensitive values are masked and should be provided through secure environment variables.

Table D.1: Deployment Configuration Groups

Group	**Example Variables**	**Notes**

Database	`POSTGRES_HOST`
	`POSTGRES_PORT`
	`POSTGRES_DB`
	`POSTGRES_USER_NAME`
	`POSTGRES_PASSWORD`
	`DATABASE_URL`
		Stores PostgreSQL connection settings. Real values must be masked.

JWT	`JWT_SECRET` `REFRESH_SECRET`	Stores access-token and refresh-token secrets.

RabbitMQ	`RABBITMQ_DEFAULT_USER`
	`RABBITMQ_DEFAULT_PASS`
	`RABBITMQ_URL`
		Stores message broker connection settings.

Redis	`REDIS_HOST`
	`REDIS_PORT`
	`REDIS_PASSWORD`
	`REDIS_DB`
		Stores cache service configuration.

Cloudinary	`CLOUDINARY_CLOUD_NAME`
	`CLOUDINARY_API_KEY`
	`CLOUDINARY_API_SECRET`
		Stores media upload configuration.

D.1 Deployment Notes

- Use masked values for secrets in the thesis and replace them with actual values in deployment.
- Store credentials in CI/CD secrets or environment configuration, not in source control.
- Change default RabbitMQ credentials before production use.
- Set JWT secrets as cryptographically secure strings.